CEBU INSTITUTE OF TECHNOLOGY
UNIVERSITY

IT342-G4
System Integration and Architecture

System Design Document (SDD)

Project Title: CleanIT

Prepared By: Macansantos, Axcelson Odchigue

Version: 1

Date: 2/27/2026
Status: Final

REVISION HISTORY TABLE

Version	Date	Author	Changes Made	Status
0.1	2/2/2026	Axcel Macansantos	Initial draft	Draft
0.2	2/2/2026	Axcel Macansantos	Overview, Scope and Features initialization	Review
0.3	2/16/2026	Axcel Macansantos	Purpose, User Journeys / Moscow (Must, Should, could, won't have), Acceptance Criteria	Review
0.4	2/18/2026	Axcel Macansantos	AC/Journey revisions	Review
0.5	2/23/2026	Axcel Macansantos	Revised Overview to Acceptance Criteria	Revised
1	2/27/2026	Axcel Macansantos	Final review and corrections	Final

TABLE OF CONTENTS

Contents

EXECUTIVE SUMMARY 4
1.0 INTRODUCTION 5
2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION 5
3.0 NON-FUNCTIONAL REQUIREMENTS 8
4.0 SYSTEM ARCHITECTURE 9
5.0 API CONTRACT & COMMUNICATION 10
Authentication Endpoints 11
User Registration 11
User Login 11
6.0 DATABASE DESIGN 13
7.0 UI/UX DESIGN 14
8.0 PLAN 17

EXECUTIVE SUMMARY

1.1 Project Overview

CleanIT is a structured home service booking platform that enables users to schedule professional gadget cleaning and maintenance services from verified technicians. The system includes a Spring Boot backend API, React web application, and Android mobile application, all integrated to manage booking workflows, technician scheduling, and service tracking consistently across platforms.

1.2 Objectives

1. Develop a fully functional service-booking MVP with authentication, service catalog, booking management, and service tracking
2. Implement a three-tier architecture using Spring Boot (back-end), React (web), and Android (Mobile)
3. Create RESTful APIs for communication between all system components
4. Design a responsive and consistent user interface across web and mobile platforms
5. Deploy backend, database, and frontend components to production-ready cloud environments

1.3 Scope

Included Features:

- User registration and authentication (Client / Technician roles)
- Government ID upload and account verification
- Service catalog with predefined base services and pricing
- Conditional add-on compatibility based on selected base service
- Dynamic price computation based on base service + allowed add-ons
- Booking creation with technician selection
- Dedicated time slot scheduling with conflict prevention
- Technician workload enforcement (1 active + 1 upcoming booking rule)
- Service lifecycle tracking:

  (Pending, Confirmed, In Progress, Completed, Cancelled, No show)
- Pre-service checklist requirement

- Before and after service photo upload
- Admin panel for verification and booking oversight
- Responsive web interface
- Native android mobile application
- Supabase-hosted PostgreSQL database with relational schema
- Offline payment handling (Cash upon completion)

Excluded Features:

- Online payment gateway integration
- Real-time GPS tracking
- Automated technician assignment
- Product-style rating and review system
- Messaging or chat system
- Inventory management automation
- Push notifications
- Analytics dashboard
- Rating and review system
- Messaging or chat system

1.0 INTRODUCTION

1.1 Purpose

This document serves as the comprehensive design specification for the CleanIT system. It provides detailed functional requirements, architectural decisions, API contracts, database design considerations and implementation guidelines to ensure all system components integrate seamlessly across web and mobile platforms

CleanIT is designed as a structured service-booking platform for home gadget maintenance, focusing on scheduling control, service tracking, and technician workload management.

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION

2.1 Project Overview

Project Name: CleanIT
Domain: Service-Booking / Home Maintenance
Primary Users:

1. Customers
2. Technicians
3. Administrators

Problem Statement: Gadget owners currently rely on informal communication methods such as chat, calls, or walk-ins to schedule maintenance services. This leads to scheduling conflicts, lack of booking transparency, technician overbooking, and absence of structured service documentation.

Technicians also face difficulties managing multiple service commitments without a system that enforces workload control.

Solution: CleanIT provides a centralized service-booking system that enables clients to schedule gadget/component cleaning services from verified technicians while enforcing structured time slot allocation and technician workload limits. The system ensures service lifecycle tracking, identity verification, and administrative oversight across web and mobile platforms.

2.2 Core User Journeys

Journey 1: First-time Customer Purchase

1. User visits web application/Mobile application
2. User Clicks “Register” and creates account
3. User uploads government ID for verification
4. Admin verifies account
5. User logs in
6. User browses service catalog
7. User selects service package and add-ons
8. User selects technician

9. User selects available time slot
10. User enters service address and instructions
11. User submits booking request
12. Booking status becomes Pending
13. Technician accepts booking
14. User receives confirmation notifications

Journey 2: Technician Service Completion

1. Technician logs in
2. Technician views pending booking requests
3. Technician accepts booking (system checks workload rule)
4. Client address becomes visible
5. Technician travels to location
6. Technician completes pre-service checklist
7. Technician uploads before-service photos
8. Technician updates status to In Progress
9. Technician completes service
10. Technician uploads after-service photos
11. Technician marks booking as Completed

Journey 3: Administrator Verification

1. Admin logs in
2. Admin reviews client ID submissions
3. Admin approves or rejects accounts
4. Admin reviews technician registration request
5. Admin monitors bookings
6. Admin reviews technician workload and system usage

Journey 4: Client Cancels Pending Booking

1. Client logs in
2. Client views booking with status = Pending
3. Client clicks “Cancel Booking”

4. System updates status to Cancelled
5. Booking is removed from technician pending list
6. Client receives cancellation confirmation

Journey 5: Technician Rejects Booking

1. Technician logs in
2. Technician views booking with status = Pending
3. Technician selects “Reject”
4. System records rejection event
5. Booking remains Pending
6. Client receives rejection notification
7. Client selects another technician for the same booking

Journey 6: A No-Show Scenario

1. Booking is Confirmed
2. Technician arrives at location
3. Client is unavailable
4. Technician selects “Mark as No-Show”
5. Booking status updates to No-Show
6. Event is logged in system

2.3 Feature List (MoSCoW)

MUST HAVE

1. User authentication (Client / Technician / Admin roles)
2. Government ID upload and verification workflow
3. Service catalog with predefined pricing
4. Booking system with technician selection
5. Time slot scheduling with conflict prevention
6. Technician workload enforcement (1 active + 1 upcoming booking)
7. Service lifecycle tracking (Pending, Confirmed, In Progress, Completed, No-Show)
8. Pre-service checklist requirement
9. Before and after photo documentation
10. Admin verification panel

SHOULD HAVE

1. Booking history for clients
2. Basic email notifications

3. Input validation feedback
4. Responsive design for all screen sizes
5. Technician availability toggle

COULD HAVE

1. Basic dashboard showing number of completed services
2. Order filtering by status
3. Basic reporting view for admin

WON'T HAVE

1. Online payment gateway integration
2. Real-time GPS tracking
3. Automated technician assignment
4. Rating and review system
5. Messaging/chat feature
6. Inventory management automation
7. Push notifications
8. Multi-company marketplace functionality

2.4 Detailed Feature Specifications

Feature: User Authentication

- **Screens:** Register Login
- **Fields:** Name, Email, Password, Contact Number, Government ID Upload.
- **Validation:** Email format, password strength, age > 17
- **API Endpoints:**
    1. POST /auth/register
    2. POST /auth/login
- **Security:** JWT tokens, password hashing using bcrypt

Feature: Service Catalog

- **Screens:** Product Listing, Product Detail
- **Display:** Service name, Description, Duration, Base Price

- **Add-ons:** Optional service enhancements conditionally available depending on selected base service
- **API Endpoints:**
    1. GET /services
    2. GET /services/{id}

Feature: Booking System

- **Screens:** Booking Form, Booking History
- **Functions:** Select Technician, Select TimeSlot, Enter service Location, Calculate total price
- **Business Rule:**
    1. Technicians can only have 1 active booking and 1 upcoming accepted booking.
    2. Technicians may reject booking only while status = Pending.
    3. Technician cannot cancel booking once status = In Progress.
    4. Cancellation after confirmation requires Admin intervention.
    5. Technician can view service location only after booking status = Confirmed.

- **API Endpoints:**
    1. POST /bookings
    2. GET /bookings
    3. PUT /bookings/{id}/accept
    4. PUT /bookings/{id}/status

-*Service Compatibility Rules*

1. Client must select exactly one base service
2. Add-ons are filtered dynamically
3. System must prevent duplicate service scope
4. Standalone base services are mutually exclusive.
5. Deep Cleaning includes internal component servicing and therefore cannot be combined with GPU Cleaning or PSU Cleaning.
6. Standalone services cannot include duplicate internal operations

-*Role-Based Action Rules*

1. Client may cancel booking only when status = Pending.
2. Client cannot modify booking once status = Confirmed.

3. Client cannot change technician, time slot, or address after confirmation.
4. Clients have read-only access for Confirmed, In Progress, Completed, Cancelled, and No-Show states.
5. Only Technician may transition Confirmed → In Progress.
6. Only Technician may transition In Progress → Completed.
7. Only Technician may mark No-Show.
8. Only Admin may override lifecycle states.

Feature: Service Lifecycle

- **Status:** Pending, Confirmed, In Progress, Completed, Cancelled, No-Show
  Allowed State Transition:
  1. Pending -> Confirmed
  2. Pending -> Cancelled
  3. Confirmed -> In Progress
  4. Confirmed -> No show
  5. In Progress -> Completed
- **Requirements:**
  Pre-service checklist completion
  Before/After photo upload
  Timestamp

Feature: Admin Panel

- **Screens:** User Verification, Booking Management
- **Functions:** Approve/Reject User Accounts, Approve/Reject technicians Registrations, View system-wide bookings
- **Access-control:** Admin role required
- **API Endpoints:**
  1. GET /admin/users
  2. PUT /admin/verify/{id}
  3. GET /admin/bookings

2.5 Acceptance Criteria

AC-1: Successful User Registration

- ➢ Given I am a new user
- ➢ When I enter valid details and upload ID

- And submit registration
- Then my account should be created
- And marked as Pending Verification
- And I cannot book services until verified

AC-2: Booking Creation

- Given I am a verified client
- When I select service, technician, and available time slot
- And submit booking
- Then booking status should be Pending
- And appear in the selected technician's dashboard

AC-3: Technician Workload Enforcement

- Given a technician already has 1 active booking
- And 1 upcoming accepted booking
- when the technician attempts to accept another booking
- Then the system must prevent acceptance
- And display a workload limit message

AC-4: Service Completion

- Given technician has marked booking as Completed
- Then booking status should update to Completed
- And client should see updated status
- And service documentation should be stored in the system

AC-5: Booking Cancelation

- Given I am a verified client
- And booking status is Pending
- When I cancel the booking
- Then status must update to Cancelled
- And technician must no longer see it as pending

AC-6: Technician Rejection

- Given I am a technician
- When I reject as pending booking
- Then booking must remain pending

- And client must receive notification

AC-7: Address Privacy Enforcement

- Given booking status is Pending
- When technician has not accepted
- Then client address must not be visible

AC-8: Time Slot Conflict Prevention

- Given a technician has already accepted a booking at a specific time slot
- When another client attempts to book the same technician and time slot
- Then the system must prevent booking
- And display slot unavailable message

AC-9: Workload Enforcement at Server Level

- Given technician already has 1 active and 1 upcoming booking
- When technician attempts to accept another booking
- Then system must reject requests at backend level
- And return workload limit error response

AC-10: Add-on Compatibility Enforcement

- Given a client selects Deep Cleaning
- When viewing add-on options
- Then GPU Cleaning and PSU Cleaning must not be selectable
- Given client selects GPU Cleaning (Standalone)
- Then Deep Cleaning must not be selectable

AC- 11: Pre-Service Checklist Enforcement

- Given booking status = Confirmed
- And technician has not completed all checklist items
- When technician attempts transition Confirmed -> In Progress
- Then system must prevent status transition
- And display checklist completion requirement

AC- 12: Photo Documentation Requirement

- Given booking status = In Progress
- When technician attempts transition In Progress -> Completed

- ➢ Then system must require at least one Before photo and one After photo
- ➢ And prevent completion if not provided

3.0 NON-FUNCTIONAL REQUIREMENTS

3.1 Performance Requirements

- API response time: ≤2 seconds for 95% of requests
- Booking creation request processing ≤2 seconds
- Web page load time: ≤ 5 seconds on broadband
- Mobile app cold start: ≤ 5 seconds
- Image upload processing time ≤ 5 seconds per file
- Support 100 concurrent active users
- Database queries executes within 500ms for standard operation
- Time slot conflict validation must be executed within 500ms

3.2 Security Requirements

- HTTPS enforced for all client-server communications
- JWT-based authentication for all protected endpoints
- Password hashing with bcrypt (salt rounds = 12)
- Role-based access control (Client, Technician, Admin)
- Cross-site script protection
- Rate limiting: 100 requests/minute per IP
- Admin endpoints require role verification
- Client address visibility only after technician acceptance
- Sensitive data stored securely in Supabase Storage
- Clients can always view their own submitted address.
- Technician can view address only after booking is Confirmed.
- Government ID uploads must be private and accessible only to Admin role.
- All lifecycle state transitions must be validated at backend level.
- Unauthorized or invalid transitions must return HTTP 409 Conflict.
- UI-triggered state changes must not be trusted without server validation.

3.3 Compatibility Requirements

- **Web Browsers:** Chrome, Firefox, Safari, Edge (latest 2 versions)
- **Android:** API Level 24+ (Android 7.0+)
- **Screen Sizes:** Mobile (360px+), Tablet (768px+), Desktop (1024px+)
- **Operating Systems:** Windows 10+, macOS 10.15+, Linux Ubuntu 20.04+

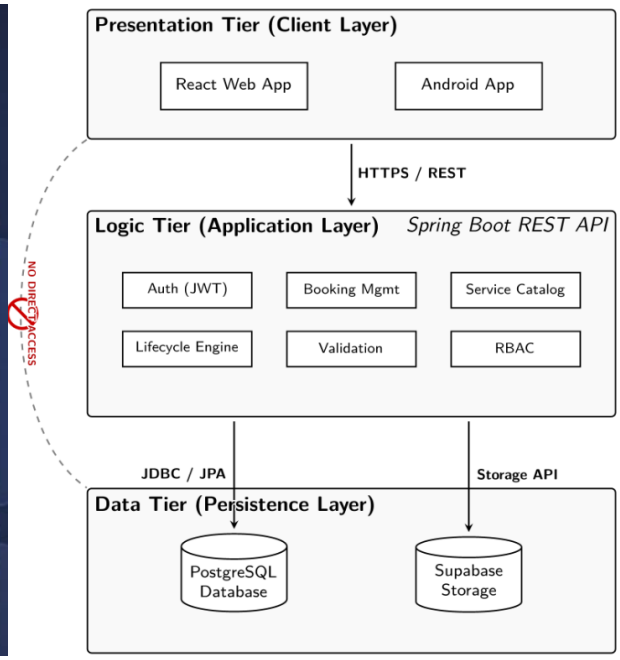
3.4 Usability Requirements

- Complete first booking within 5 minutes for new users
- Clear 4-step service tracking indicator
- Consistent navigation across web and mobile platforms
- Clear validation and error feedback messages
- Touch targets minimum 44x44 pixels on mobile
- Keyboard navigation support for web interface
- Status changes must be visually distinguishable

4.0 SYSTEM ARCHITECTURE

4.1 Component Diagram

Note: This should be a component diagram



Communications Flows:

Client HTTPS -> Spring Boot API -> JDBC/JPA

PostgreSQL -> Spring Boot API -> Supabase Storage API (File handling)

:No direct frontend to database communication

Technology Stack:

- **Backend:**

1. **Language:** Java 17
2. **Framework:** Spring Boot 3.x
3. **Security:** Spring Security (JWT Authentication)
4. **Persistence:** Spring Data JPA, Hibernate
5. **Validation:** Jakarta Bean Validation
6. **Database Driver:** PostgreSQL
7. **Deployment:** Railway / Render

- **Database:**
    1. **Provider: Supabase Cloud**
    2. **Engine: PostgreSQL 14+ (Managed)**
    3. **Features: Relational Schema, Foreign Key Constraints, Indexing, Automated Backups**
    4. **File Storage:** Supabase Storage (Government ID uploads, Before/After service photos)

- **Web Frontend:**
    1. **Library:** React 18 (Latest Stable)
    2. **Language:** TypeScript
    3. **Styling:** Tailwind CSS
    4. **HTTP Client:** Axios
    5. **Deployment Platform:** Render

- **Mobile:**
    1. **Platform:** Android
    2. **Language:** Kotlin
    3. **Deployment:** APK

- **Build Tools:**
    1. **Version Control:** GitHub
    2. **Backend Build Tool:** Maven
    3. **Web Dependency Management:** npm / yarn
    4. **Mobile Build Tool:** Gradle

5.0 API CONTRACT & COMMUNICATION

5.1 API Standards

Base URL	https://<render-backend-domain>/api/v1
Format	JSON for all requests/responses
Authentication	Authorization: Bearer <token>

Response Structure	{
	"success": boolean,
	"data": object | null,
	"error": {
	"code": string,
	"message": string,
	"details": object | null
	} | null,
	"timestamp": string
	}

5.2 Endpoint Specifications

Authentication Endpoints

User Registration

Description	User Registration
API URL	/api/v1/auth/register
HTTP Request Method	POST
Format	JSON
Authentication	None

Request Payload	{
	"name": "Juan Dela Cruz",
	"email": "juan@email.com",
	"password": "P@ssw0rd123!",
	"contactNumber": "+63 9xx xxx xxxx",
	"role": "CLIENT",
	"govIdPath": "ids/juan-delacruz.png"
	}

Response Structure	{
	"success": true,
	"data": {
	"user": {
	"id": "usr_123",
	"name": "Juan Dela Cruz",
	"email": "juan@email.com",
	"contactNumber": "+63 9xx xxx xxxx",

	"role": "CLIENT", "verificationStatus": "PENDING", "createdAt": "2026-03-03T10:00:00Z" } }, "error": null, "timestamp": "2026-03-03T10:00:00Z" }

User Login

Description	User Login
API URL	/auth/login
HTTP Method	POST
Format	JSON for all requests/responses
Authentication	None

Request Payload	{
	"name": "Juan Dela Cruz",
	"email": "juan@email.com",
	"password": "P@ssw0rd123!",
	"contactNumber": "+639123456789",
	"role": "CLIENT"
	}

Response Structure	{
	"success": true,
	"data": {
	"user": {
	"id": "usr_123",
	"name": "Juan Dela Cruz",
	"email": "juan@email.com",
	"contactNumber": "+639123456789",
	"role": "CLIENT",
	"verificationStatus": "PENDING",
	"createdAt": "2026-03-03T10:00:00Z"
	}
	},
	"error": null,
	"timestamp": "2026-03-03T10:00:00Z"
	}

5.3 Error Handling

HTTP Status Codes

- 200 OK - Successful request
- 201 Created - Resource created
- 400 Bad Request - Invalid input
- 401 Unauthorized - Authentication required/failed
- 403 Forbidden - Insufficient permissions
- 404 Not Found - Resource doesn't exist
- 409 Conflict - Duplicate resource
- 500 Internal Server Error - Server error

Error Code Examples

Invalid LOGIN	{
	"success": false,
	"data": null,
	"error": {
	"code": "AUTH-001",
	"message": "Invalid email or password.",
	"details": null
	},
	"timestamp": "2026-03-03T10:30:00Z"
	}

Validation Fail	{
	"success": false,
	"data": null,
	"error": {
	"code": "VALID-001",
	"message": "Validation failed.",
	"details": {
	"email": "Email is required.",
	"password": "Password must be at least 8 characters."
	}
	},
	"timestamp": "2026-03-03T10:30:00Z"
	}

Common Error Codes

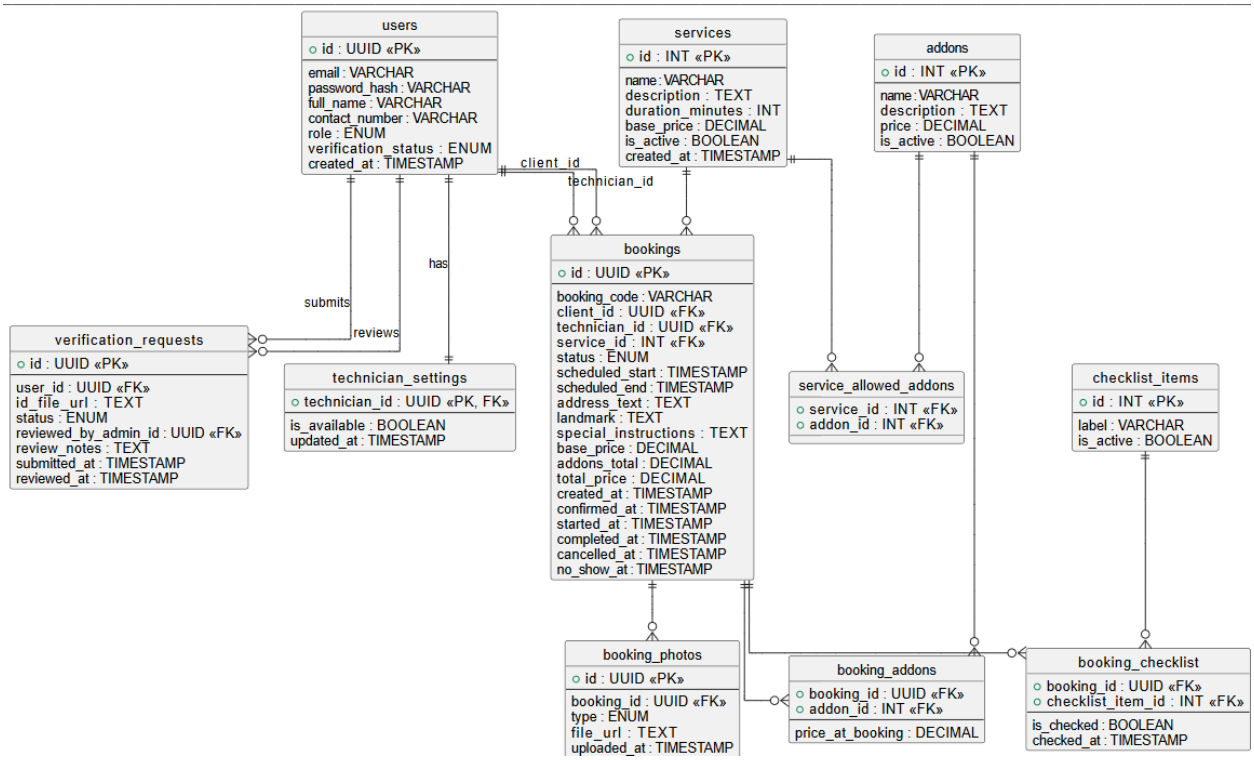
- AUTH-001: Invalid credentials

- AUTH-002 – Token expired or invalid
- VALID-001 – Missing required fields
- VALID-002 – Invalid format (email, phone).
- VERIFY-001 – Account not verified (booking blocked)
- VERIFY-002 – ID file missing.
- BOOK-001 – Booking not found
- BOOK-002 – Booking already cancelled/completed.
- SLOT-001 – Time slot unavailable.

6.0 DATABASE DESIGN

6.1 Entity Relationship Diagram

Note: This should be an ERD (Database Schema)



Detailed Relationships:

Users

- **User (Client) 1 → Bookings 1:N**
- **User (Technician) → Bookings → 1:N**
- **User (Admin) → VerificationRequests (reviewed) → 1:N**
- **User → VerificationRequests (submitted) → 1:N**
- **User (Technician) ↔ TechnicianSettings → 1:1**

Services & Add-ons

- **Service** → Bookings → 1:N
- **Service** ↔ AddOns → M:N (via service_allowed_addons)

Bookings

- **Booking** ↔ AddOns → M:N (via booking_addons)
- **Booking** ↔ ChecklistItem → M:N (via booking_checklist)
- **Booking** → BookingPhotos → 1:N

Verification

- **One-to-One:** User ↔ Cart (Each user has exactly one cart)
- **One-to-Many:** User → Orders (User can have multiple orders)

Key Tables:

1. **users** – accounts + roles + verification status
2. **verification_requests** – gov ID submission workflow
3. **services** – base service catalog
4. **addons** – available add-ons
5. **service_allowed_addons** – which add-ons are allowed per base service
6. **bookings** – service appointment record
7. **booking_addons** – add-ons selected for the booking
8. **checklist_items** – master checklist template
9. **booking_checklist** – checklist completion per booking
10. **booking_photos** – before/after photo records
11. **technician_settings** – availability toggle + optional notes

Table Structure Summary:

- **users:**
    1. id (PK, UUID)
    2. email (unique)
    3. password_hash
    4. full_name
    5. contact_number
    6. role (CLIENT | TECHNICIAN | ADMIN)
    7. verification_status (PENDING | VERIFIED | REJECTED)
    8. created_at

- **Services:**
    1. id (PK)
    2. name (External Cleaning, Deep Cleaning, GPU Cleaning, PSU Cleaning)
    3. description
    4. duration_minutes
    5. base_price
    6. is_active
    7. created_at

- **Add-ons:**
    1. id (PK)
    2. name (Thermal Paste, Cable Management, etc.)
    3. description
    4. price
    5. is_active
- **service_allowed_addons:**
    1. service_id (FK → services.id)
    2. addon_id (FK → addons.id)
    3. PRIMARY KEY(service_id, addon_id)

- **technician_settings:**
    1. technician_id (PK/FK → users.id)
    2. is_available (boolean)
    3. updated_at

- **bookings:**
    1. id (PK, UUID)
    2. booking_code (unique like BK-2024-00123)
    3. client_id (FK → users.id)
    4. technician_id (FK → users.id, nullable until accepted if you want)
    5. service_id (FK → services.id)
    6. status (PENDING | CONFIRMED | IN_PROGRESS | COMPLETED | CANCELLED | NO_SHOW)
    7. scheduled_start (datetime)
    8. scheduled_end (datetime or store duration + compute)
    9. address_text
    10. landmark (nullable)
    11. special_instructions (nullable)
    12. base_price

13. addons_total
14. total_price
15. created_at
16. confirmed_at (nullable)
17. started_at (nullable)
18. completed_at (nullable)
19. cancelled_at (nullable)
20. no_show_at (nullable)

- **booking_addons:**
    1. booking_id (FK → bookings.id)
    2. addon_id (FK → addons.id)
    3. price_at_booking (important for history even if addon price changes later)
    4. PRIMARY KEY(booking_id, addon_id)

- **checklist_items:**
    1. id (PK)
    2. label
    3. is_active

- **booking_checklist:**
    1. booking_id (FK → bookings.id)
    2. checklist_item_id (FK → checklist_items.id)
    3. is_checked (boolean)
    4. checked_at (nullable)
    5. PRIMARY KEY(booking_id, checklist_item_id)

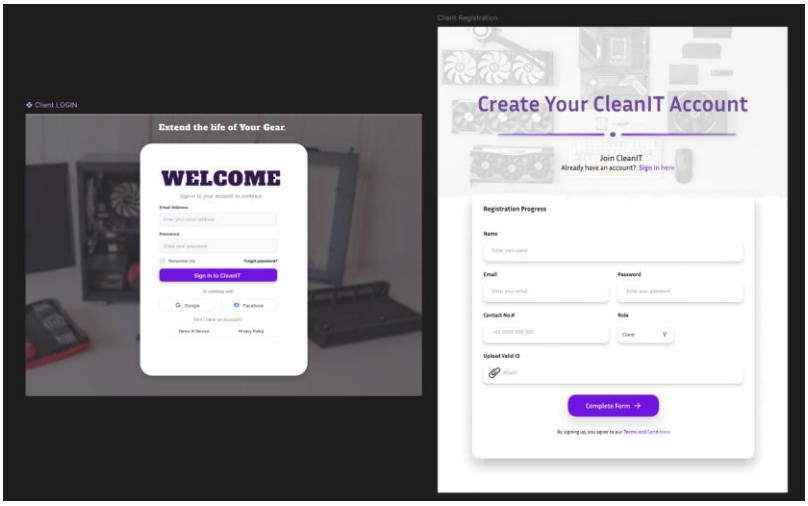
- **booking_photos:**
    1. id (PK, UUID)
    2. booking_id (FK → bookings.id)
    3. type (BEFORE | AFTER)
    4. file_url (Supabase Storage path)
    5. uploaded_at

7.0 UI/UX DESIGN

7.1 Web Application Wireframes

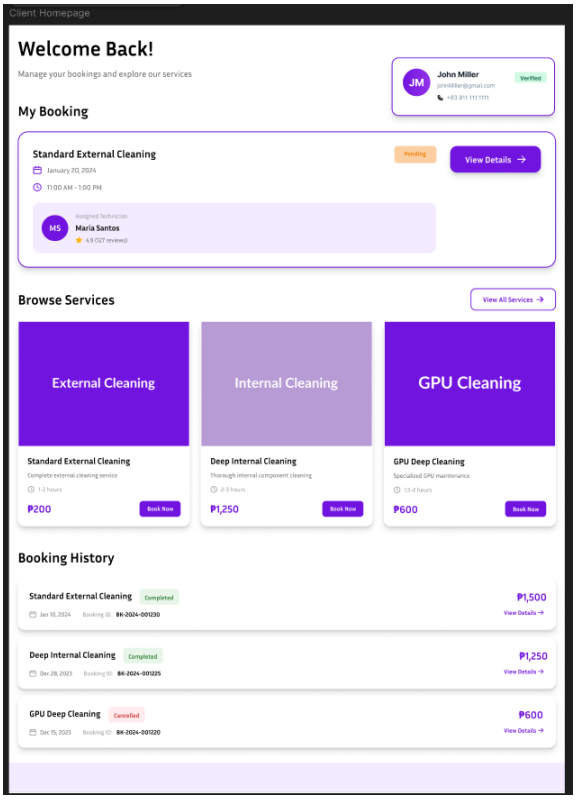
Note: This should be wireframes from Figma

CLIENT Login/Register



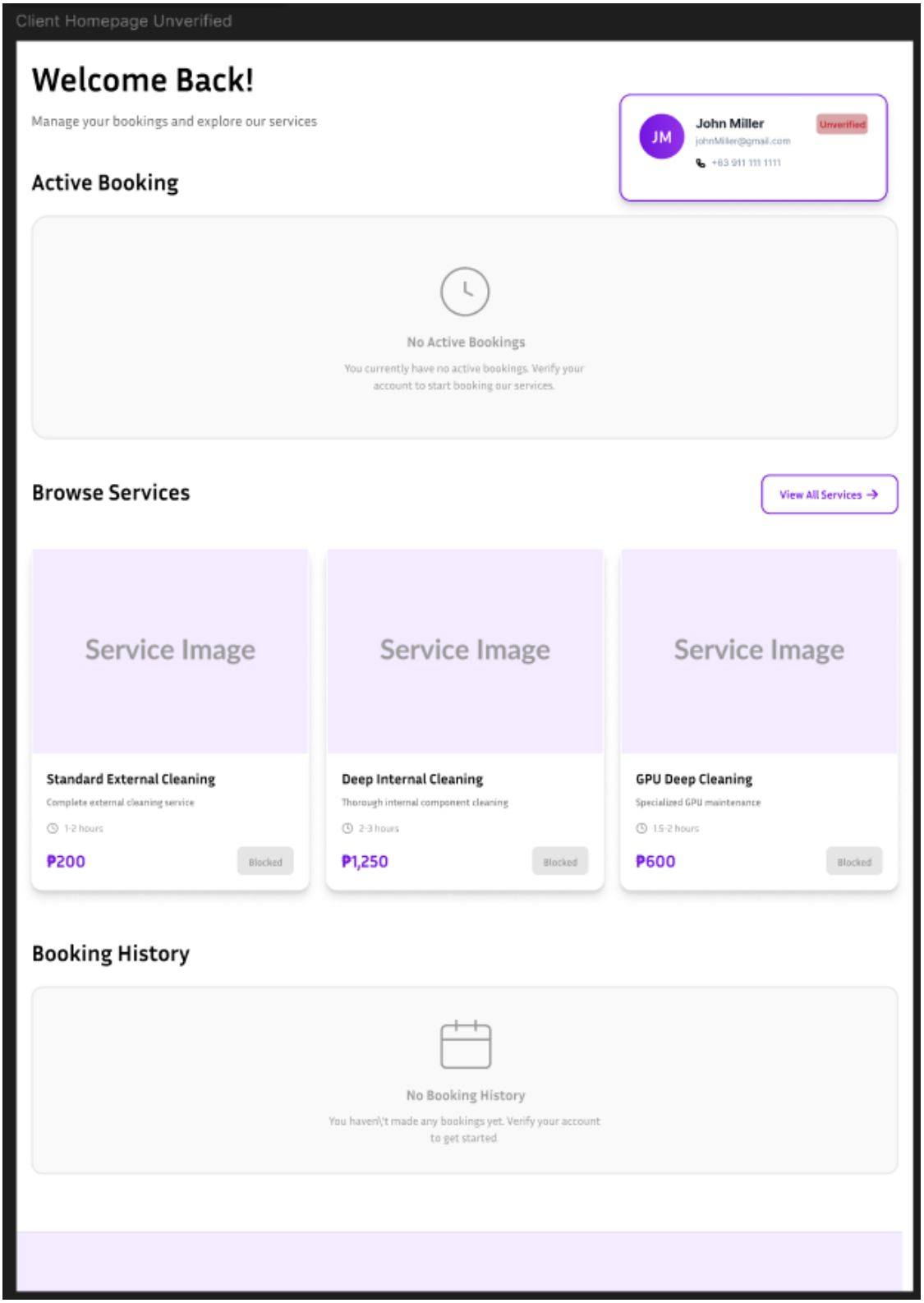
CLIENT HOMEPAGE (Verified)

-My Booking / Browse Services / Booking History



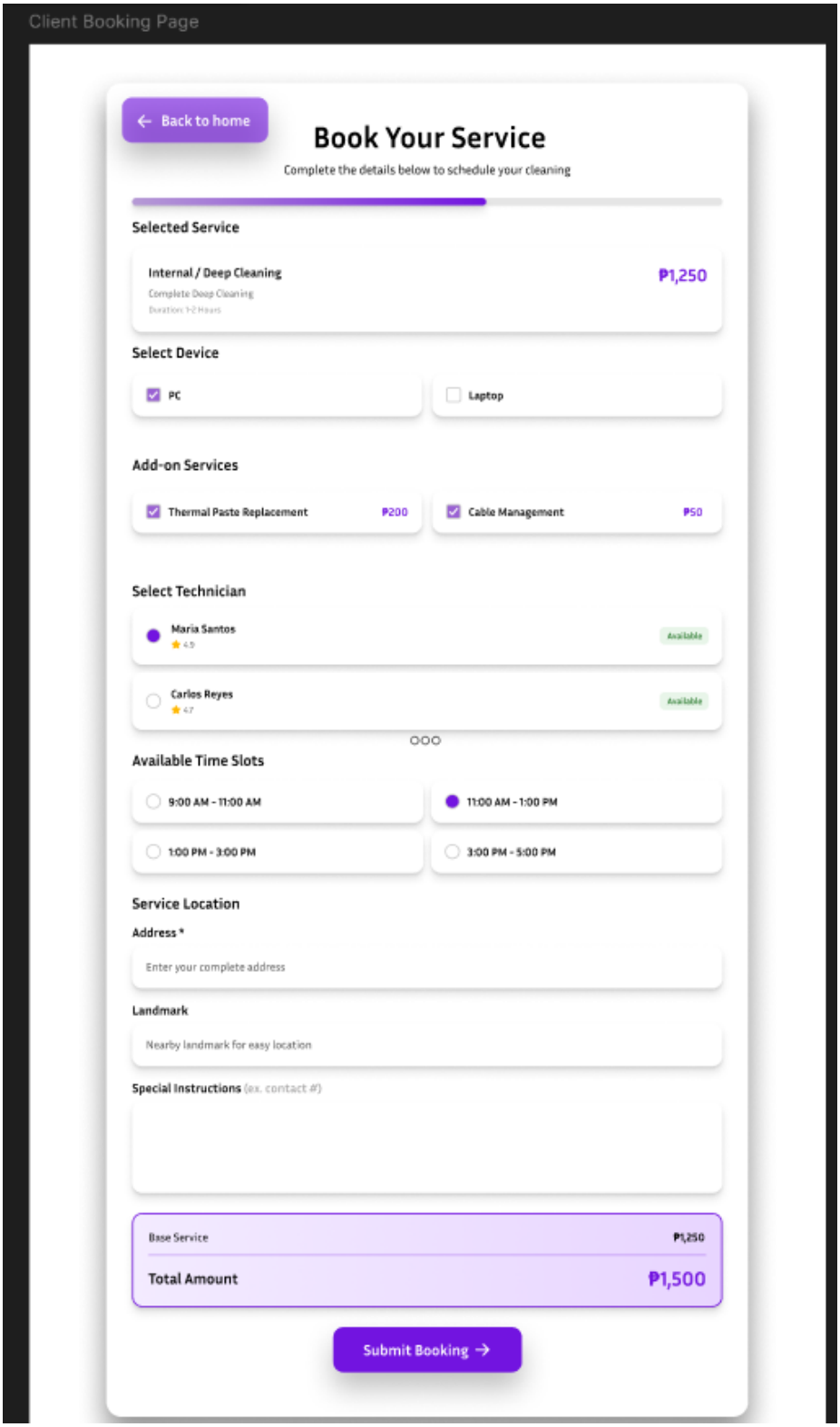
CLIENT HOMEPAGE (Unverified)

-My Booking *locked* / Browse Services *locked* / Booking History *locked*



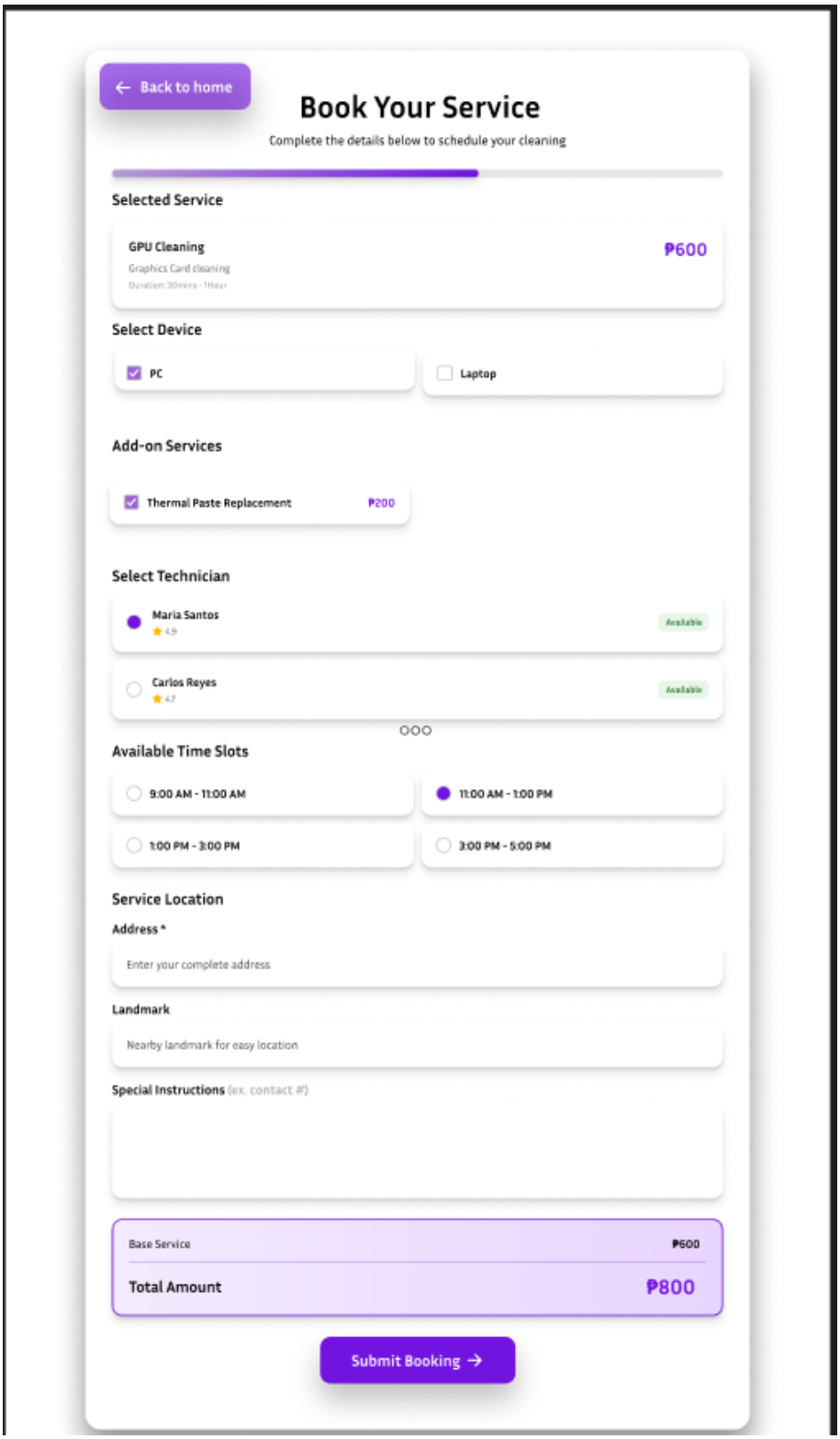
CLIENT BOOKING Page (Deep Cleaning)

-Services / Device / Add-ons / Technician / Timeslots / Location & Info / Amount



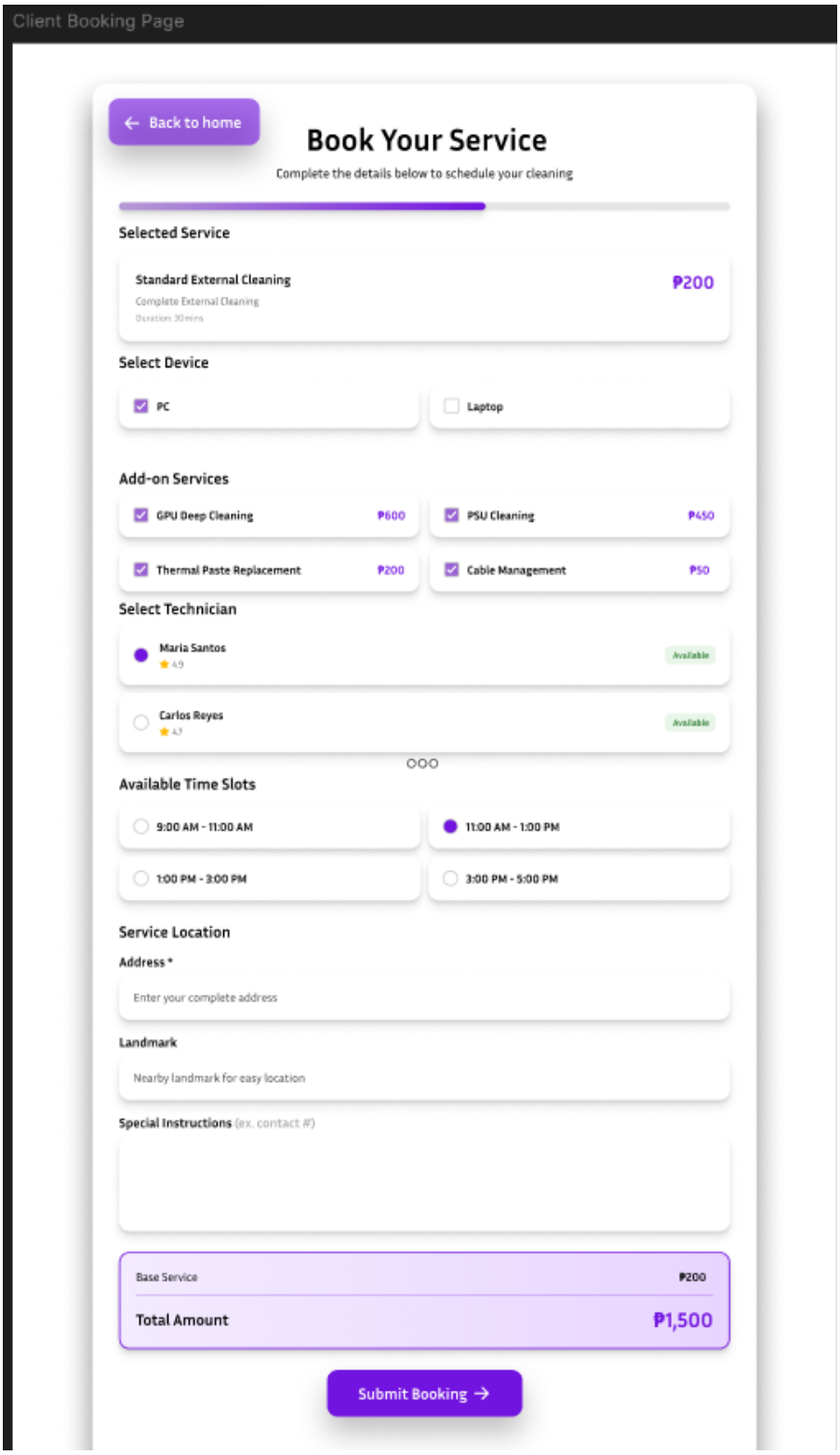
CLIENT BOOKING Page (GPU Cleaning)

-Services / Device / Add-ons / Technician / Timeslots / Location & Info / Amount



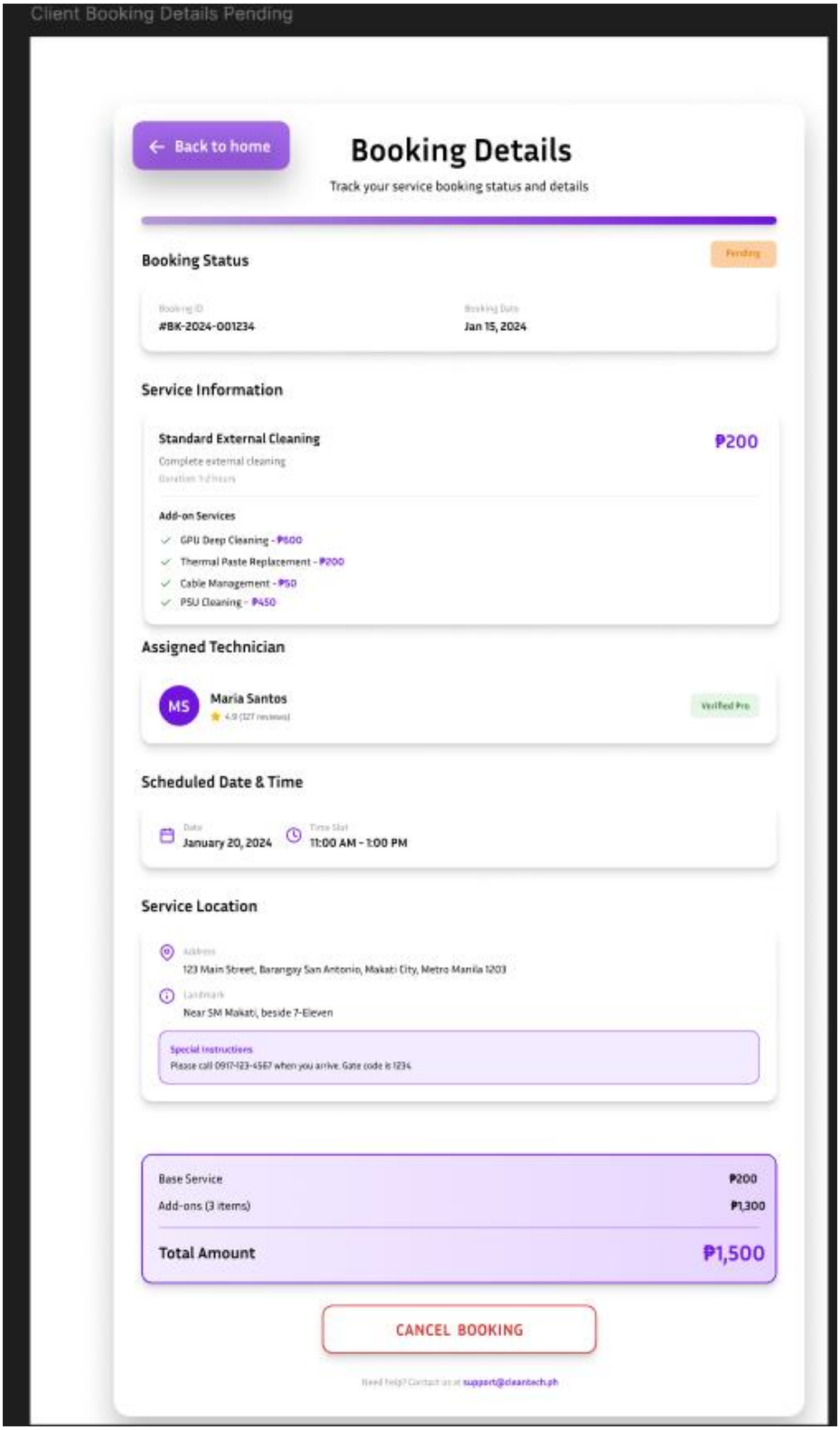
CLIENT BOOKING Page (PSU Cleaning)

-Services / Device / Add-ons / Technician / Timeslots / Location & Info / Amount



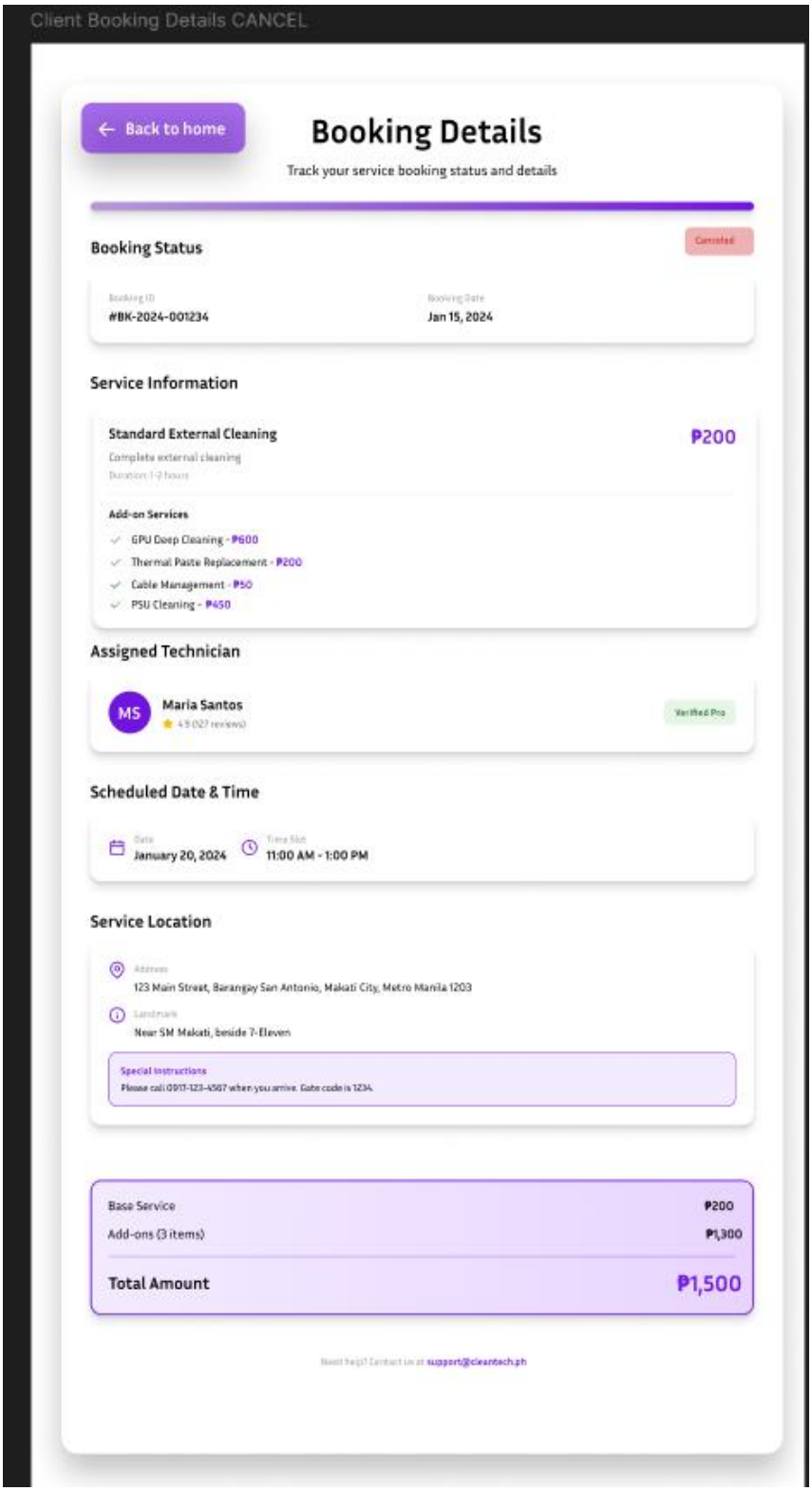
CLIENT Booking DETAILS Page (Pending)

-Status / Service & Info / Technician & Timeslots / Location & Info / Amount



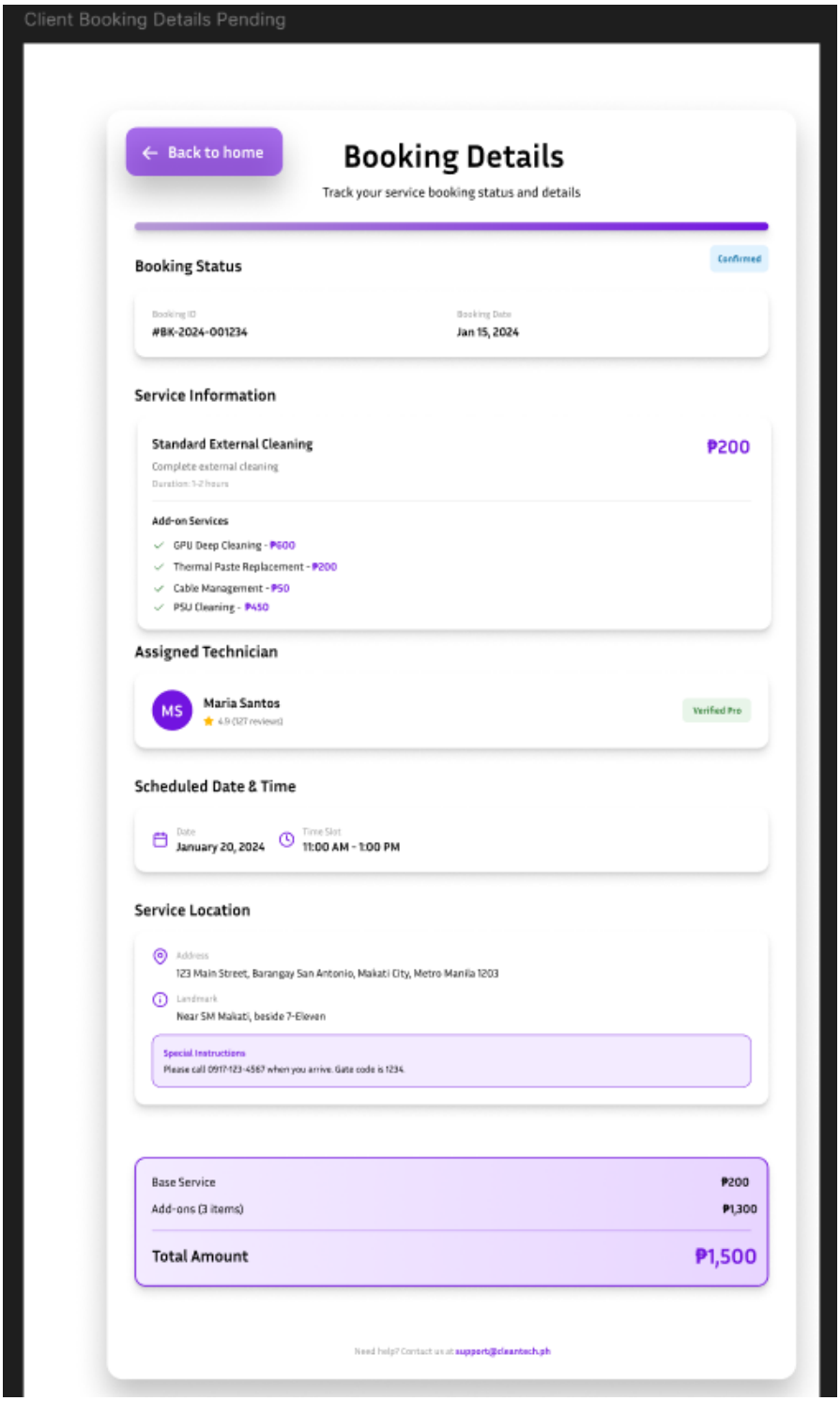
CLIENT Booking DETAILS Page (Canceled)

-Status / Service & Info / Technician & Timeslots / Location & Info / Amount



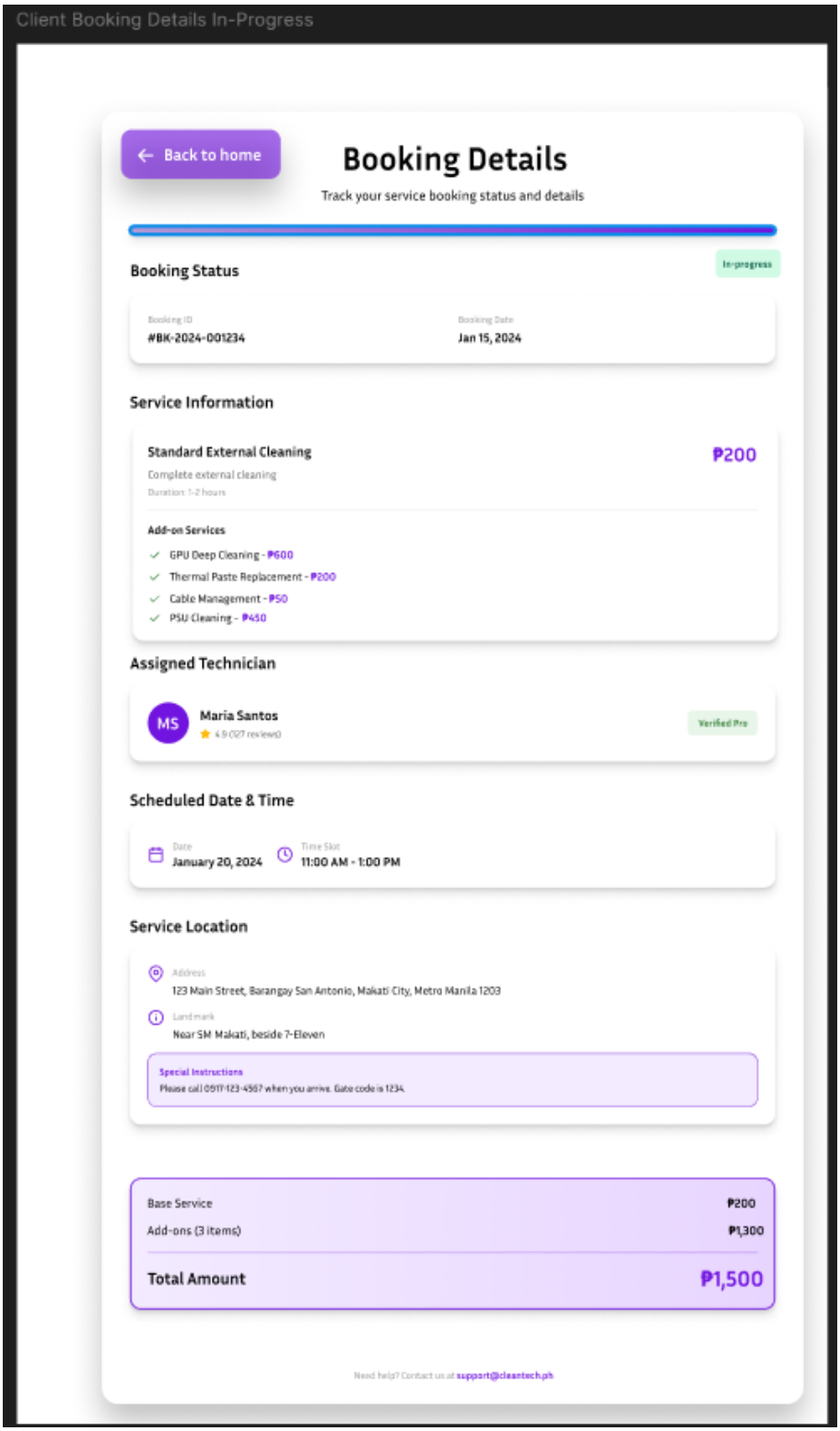
CLIENT Booking DETAILS Page (Confirmed)

-Status / Service & Info / Technician & Timeslots / Location & Info / Amount



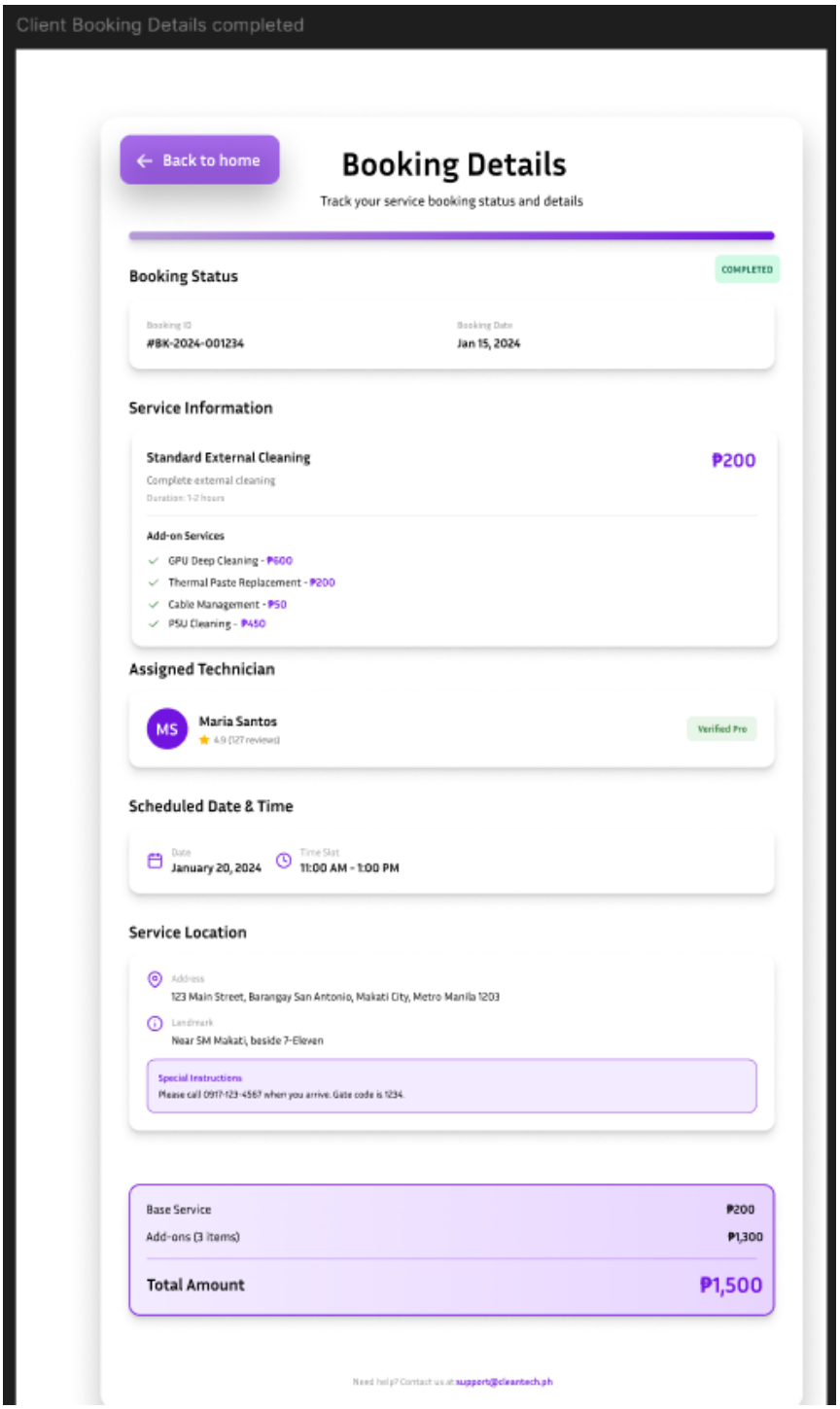
CLIENT Booking DETAILS Page (In Progress)

-Status / Service & Info / Technician & Timeslots / Location & Info / Amount



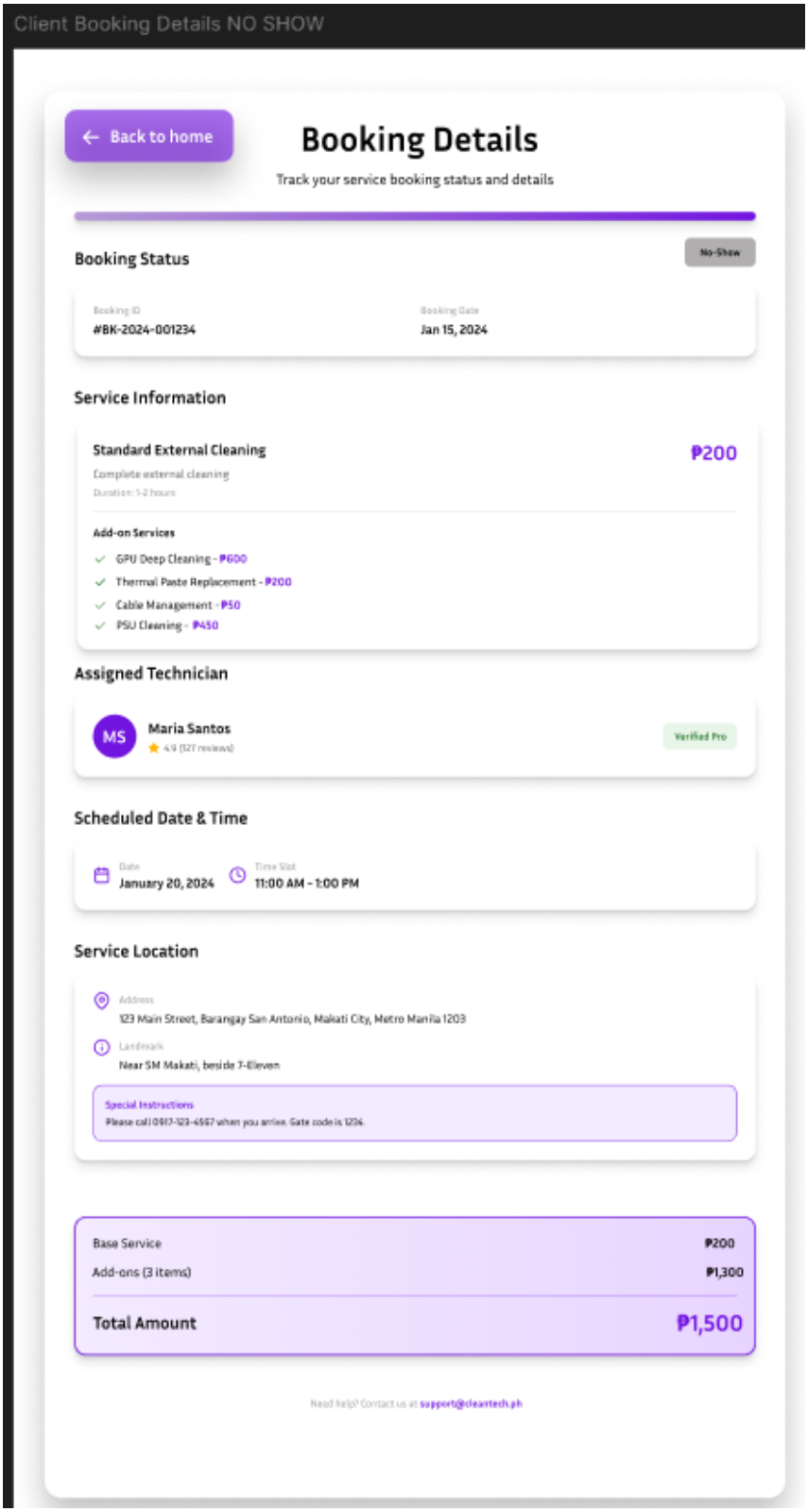
CLIENT Booking DETAILS Page (Completed)

-Status / Service & Info / Technician & Timeslots / Location & Info / Amount

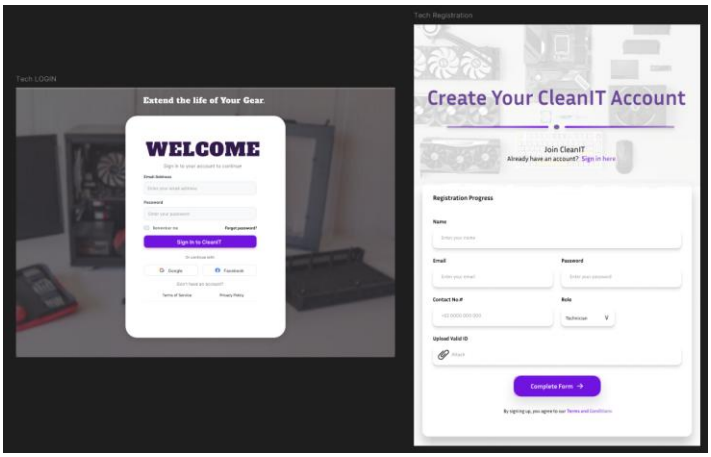


CLIENT Booking DETAILS Page (Completed)

-Status / Service & Info / Technician & Timeslots / Location & Info / Amount

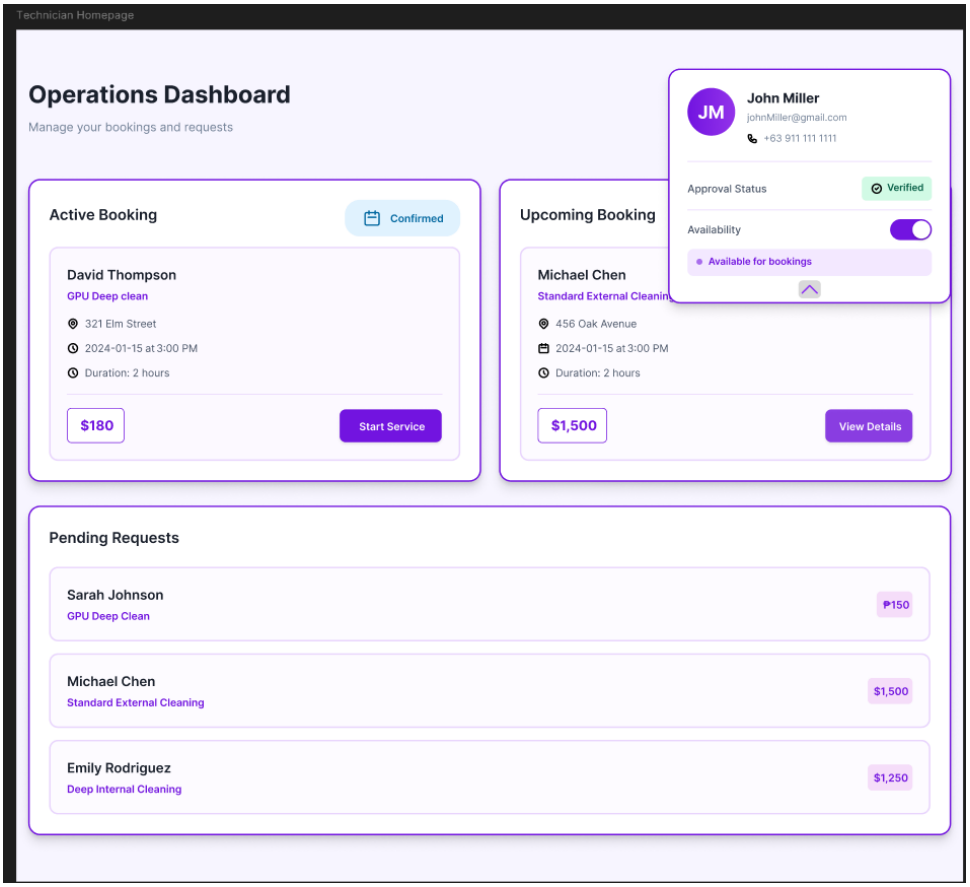


TECHNICIAN Login/Register



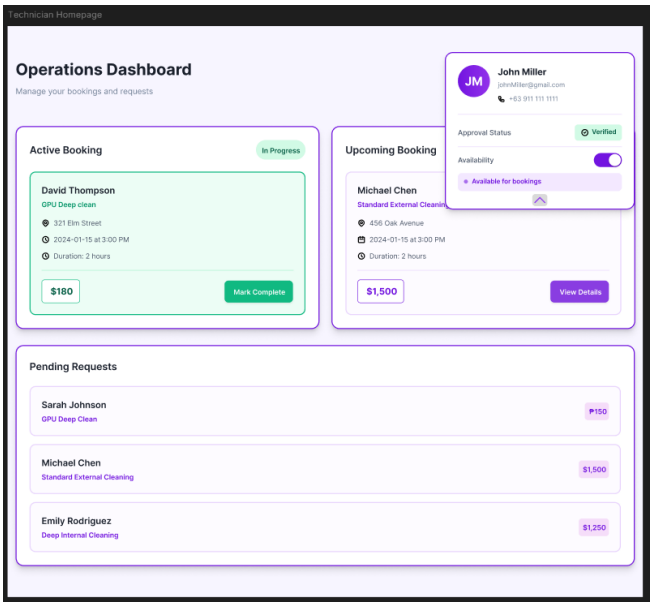
TECHNICIAN Homepage (Confirmed active)

-Active Booking / Upcoming Booking / Pending Request



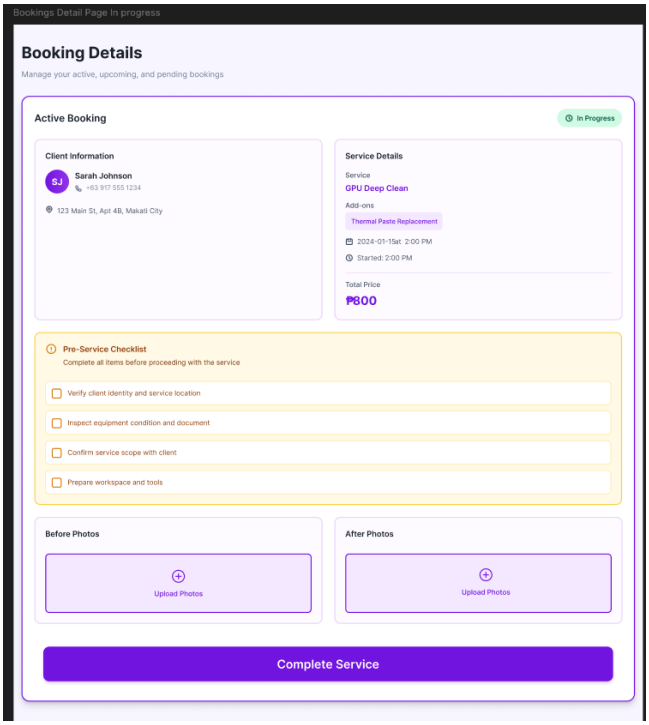
TECHNICIAN Homepage (In Progress)

-Active Booking / Upcoming Booking / Pending Request



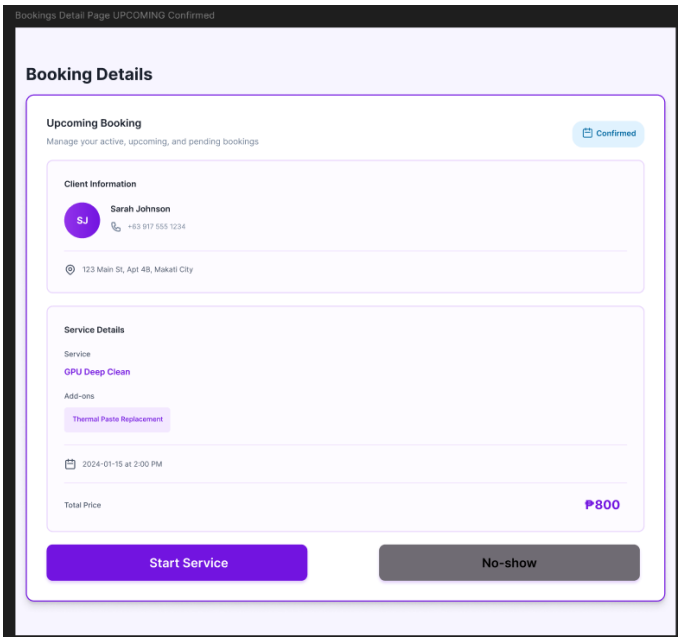
TECHNICIAN Booking DETAIL Page (In Progress)

-Active Booking / Checklist / Before After Photos / Complete Service



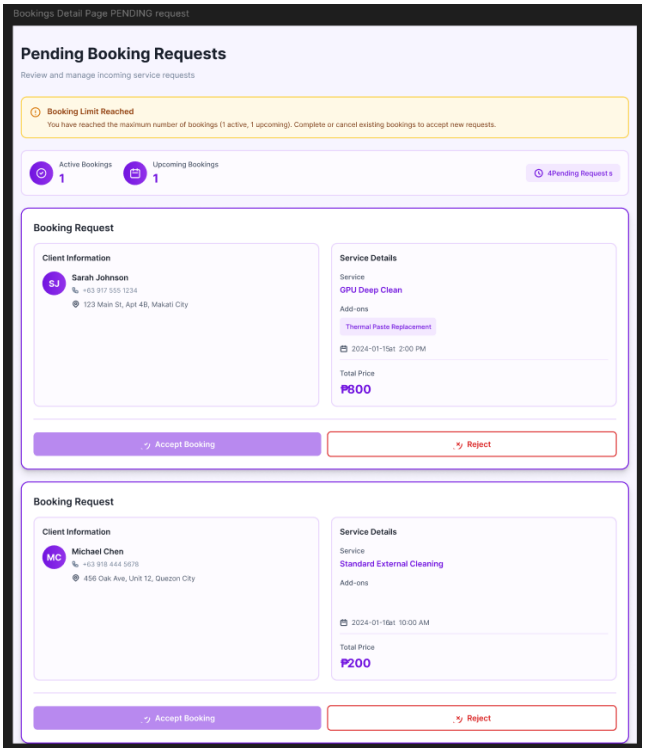
TECHNICIAN Booking DETAIL Page (UPCOMING - confirmed)

-Booking Info / Service Details / Mark as Start service or No-Show/

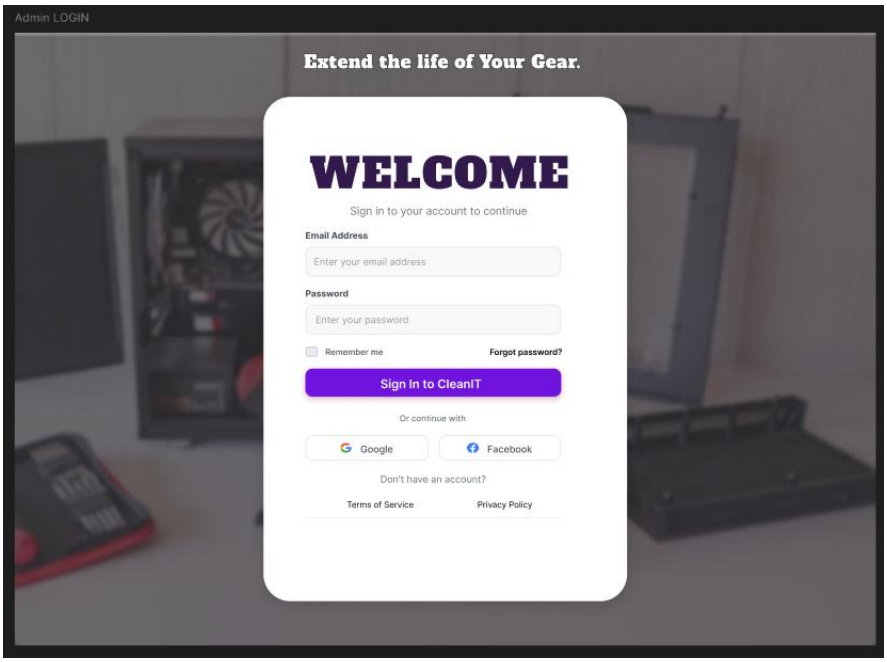


TECHNICIAN Bookings DETAIL Page (Pending Request)

-Booking Info / Service Details / Accept OR Reject

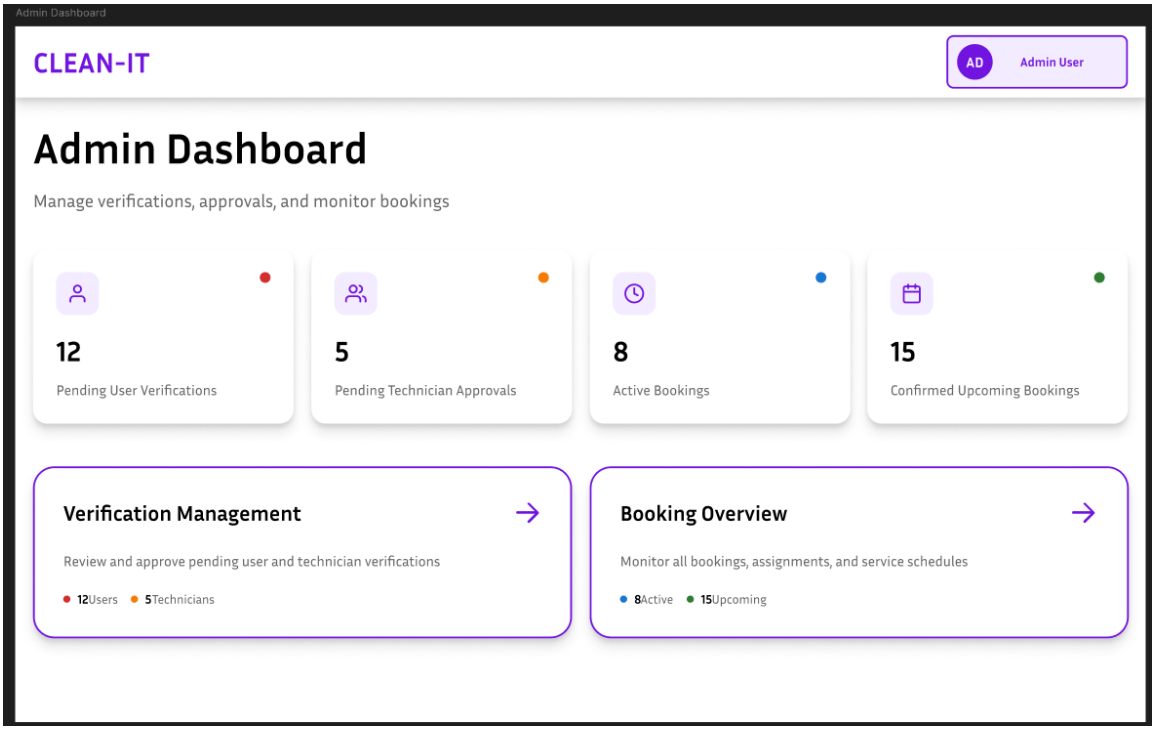


ADMIN Login



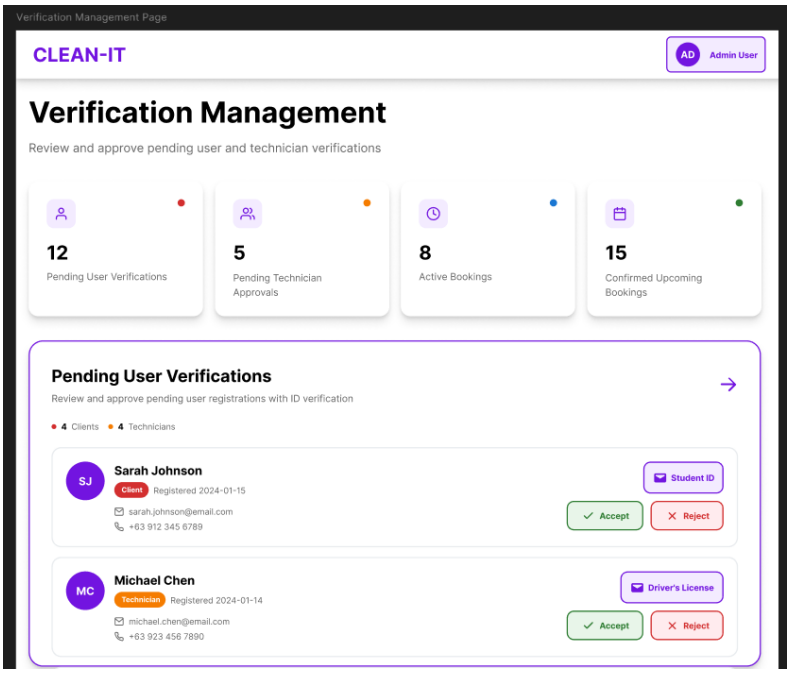
ADMIN Homepage / Dashboard

-Monitoring / Verification Management / Booking Overview



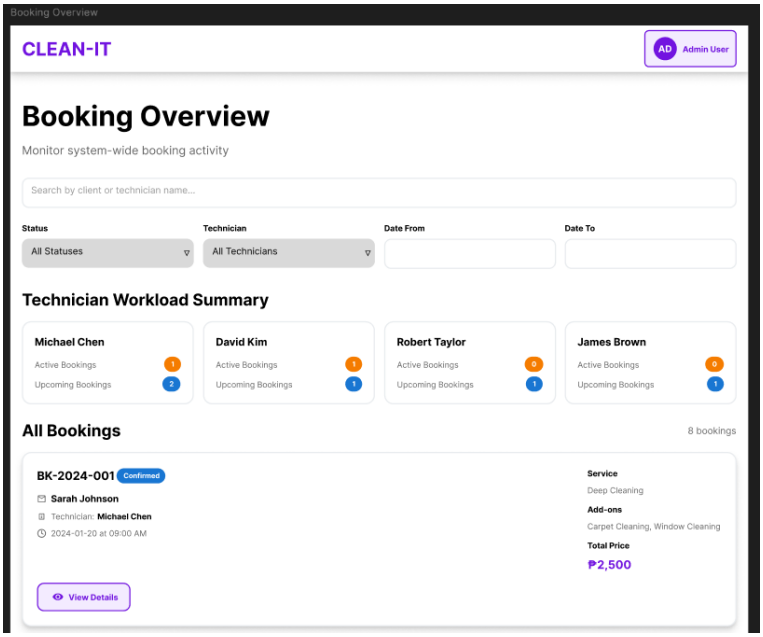
ADMIN Verification Management

-Pending User Verifications



ADMIN Booking Overview

-All Bookings

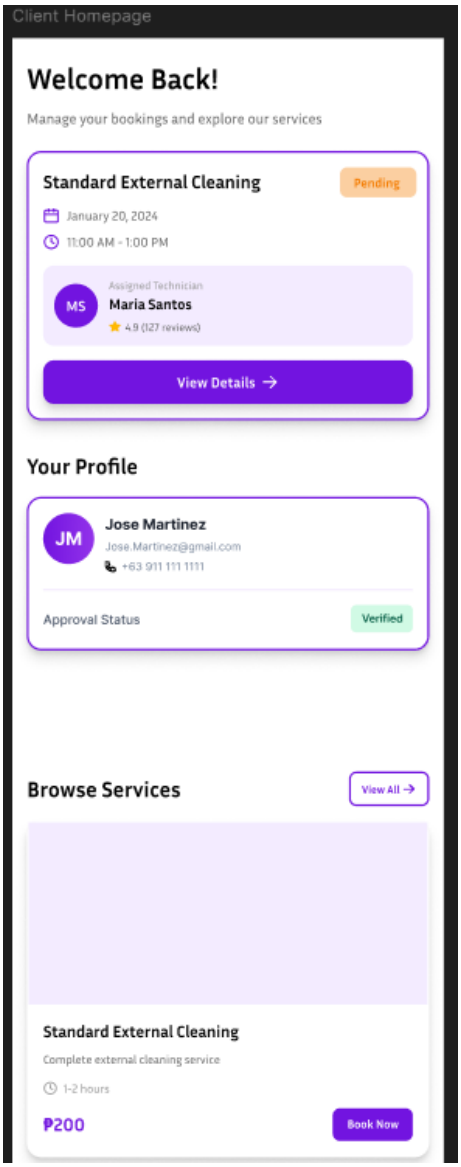


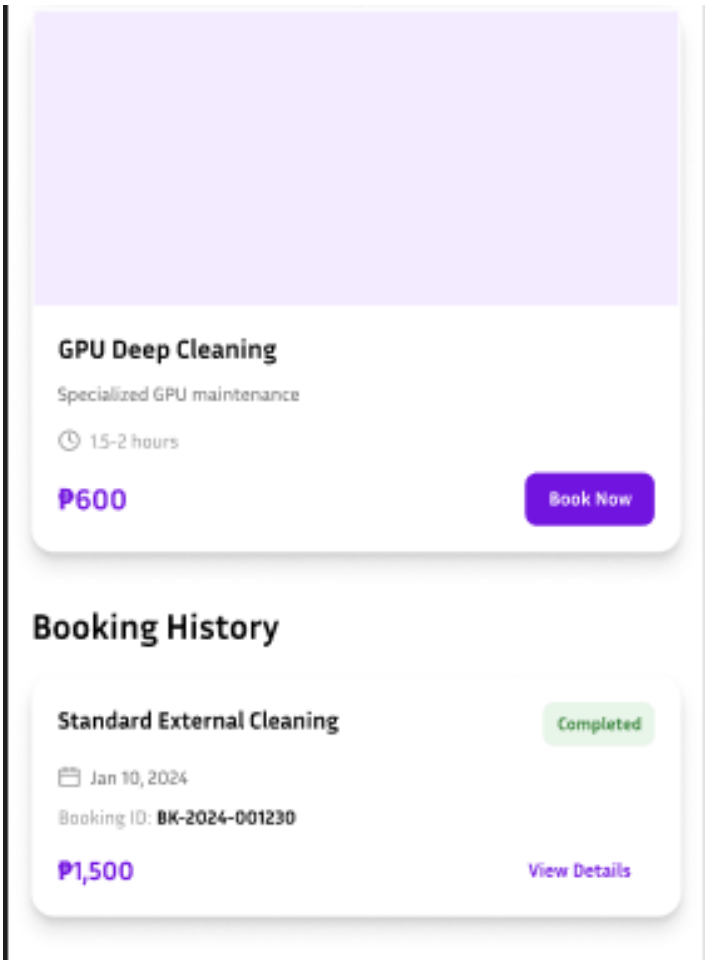
7.2 Mobile Application Wireframes

Note: This should be wireframes from Figma

CLIENT HOMEPAGE (Verified)

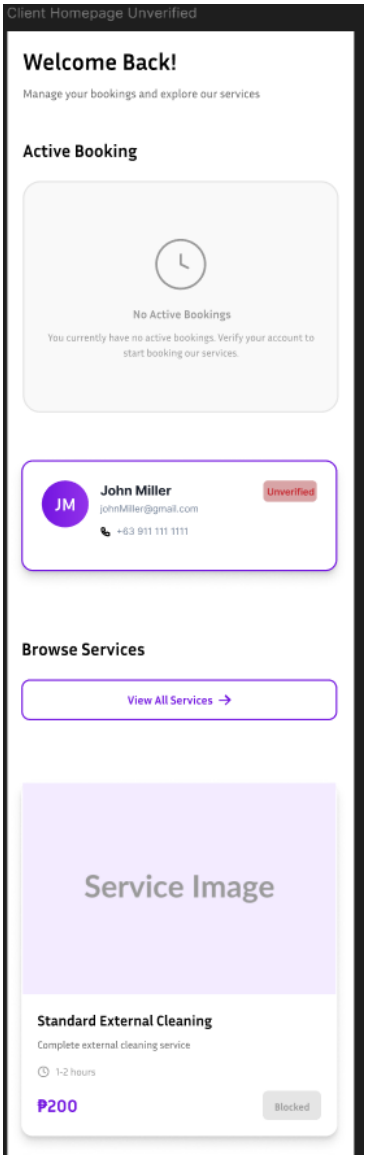
-My Booking / Browse Services / Booking History

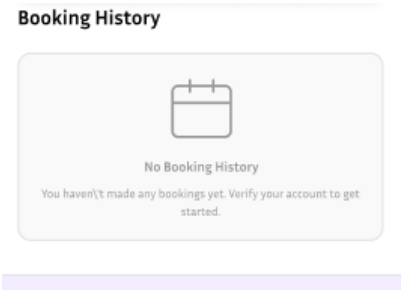




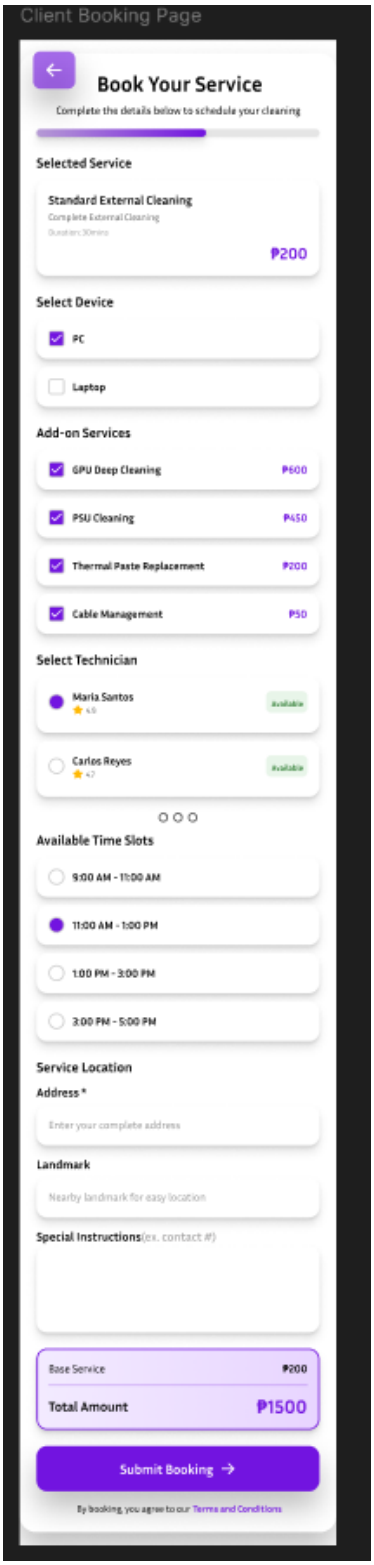
CLIENT HOMEPAGE (Unverified)

-My Booking / Browse Services / Booking History

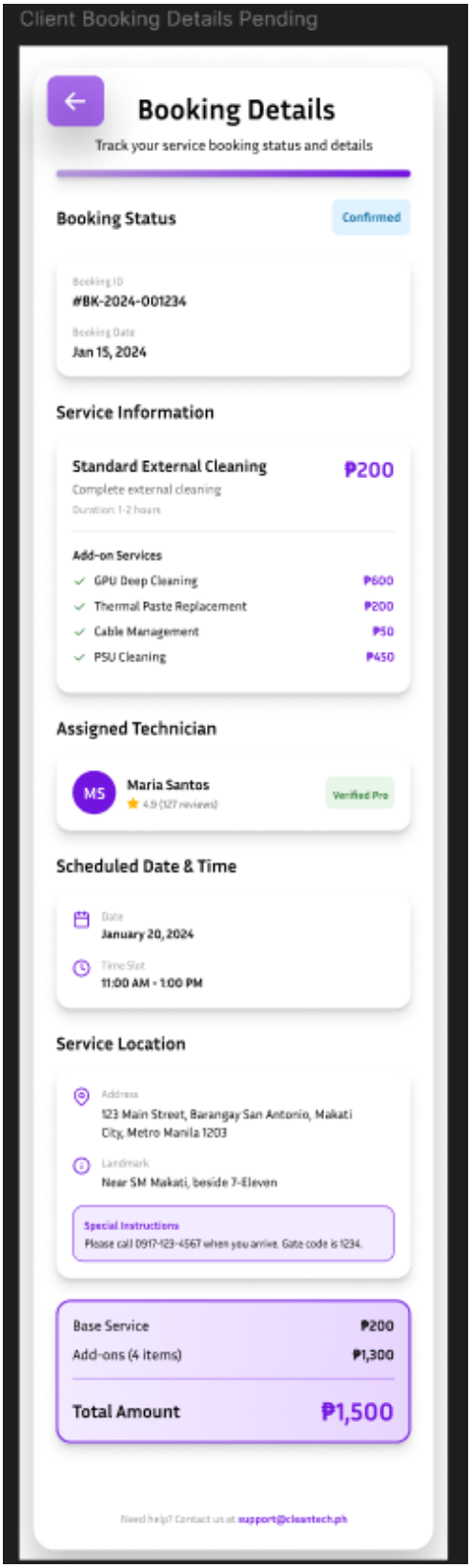




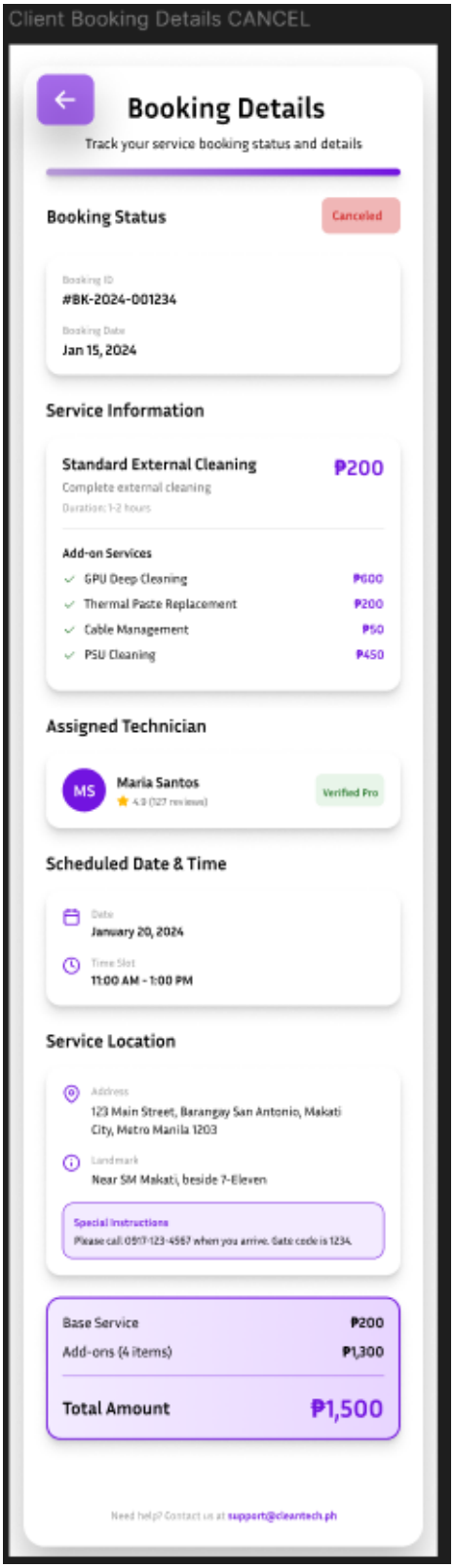
CLIENT BOOKING Page (Standard Cleaning)



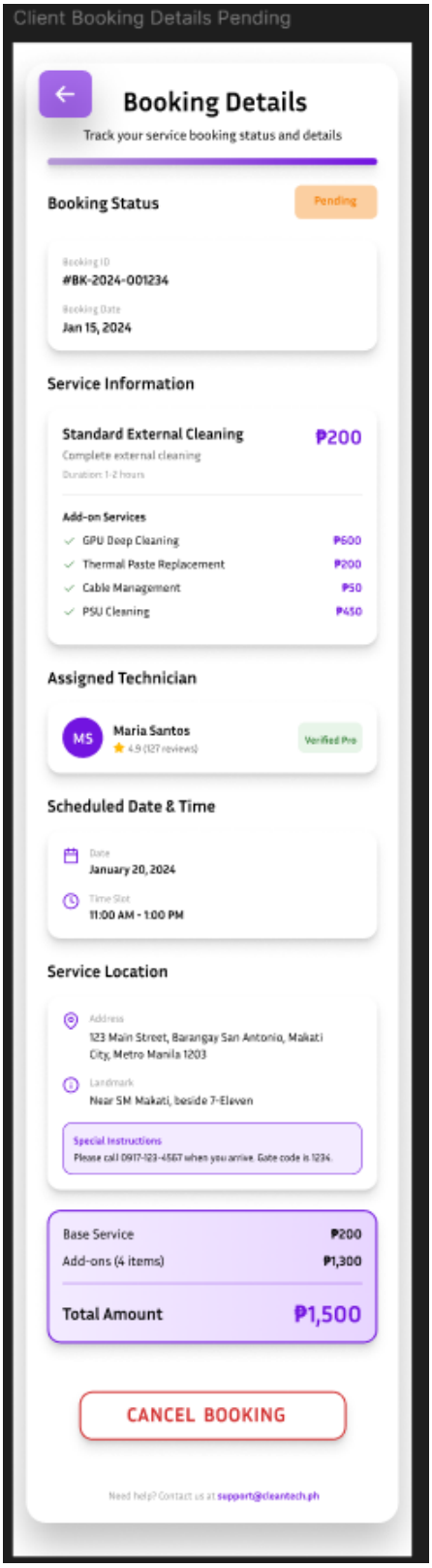
CLIENT Booking DETAILS Page (Confirm)



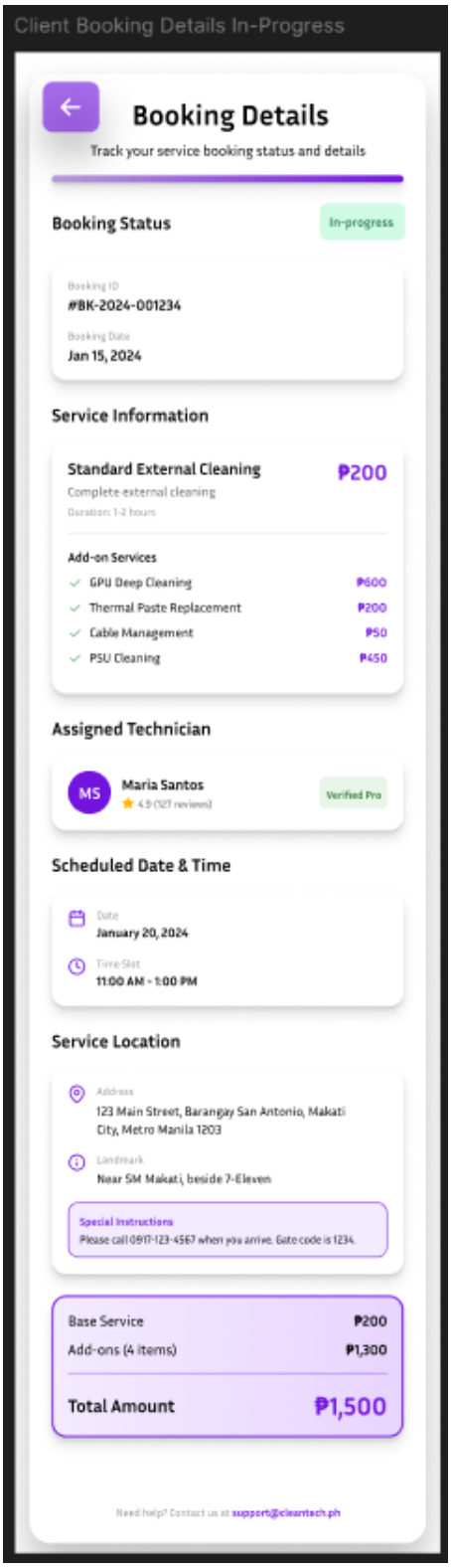
CLIENT Booking DETAILS Page (Canceled)



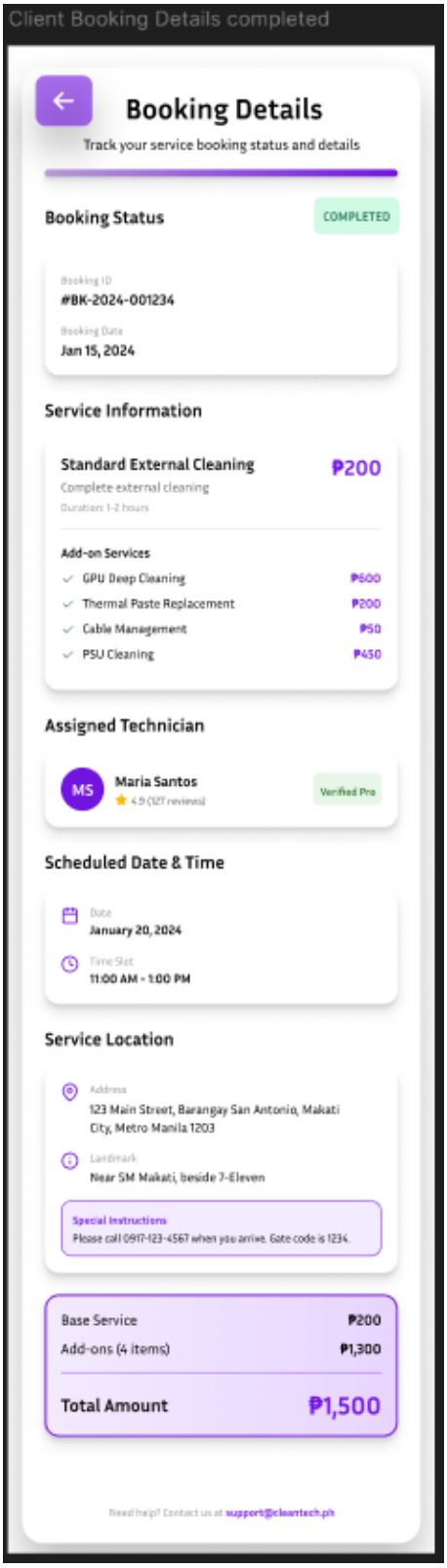
CLIENT Booking DETAILS Page (Pending)



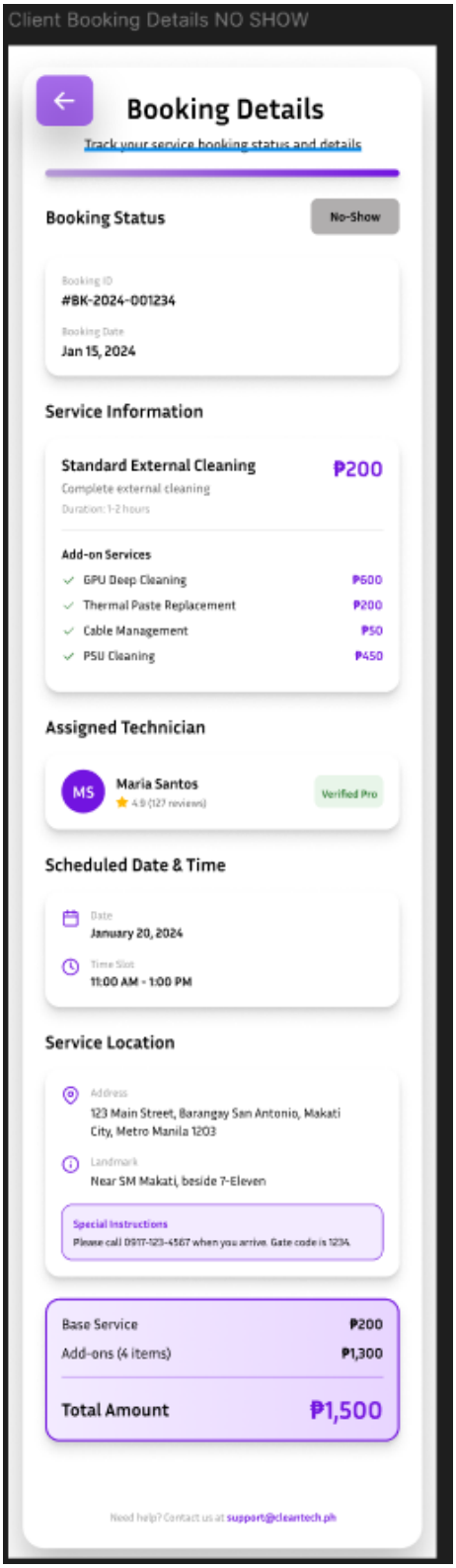
CLIENT Booking DETAILS Page (In Progress)



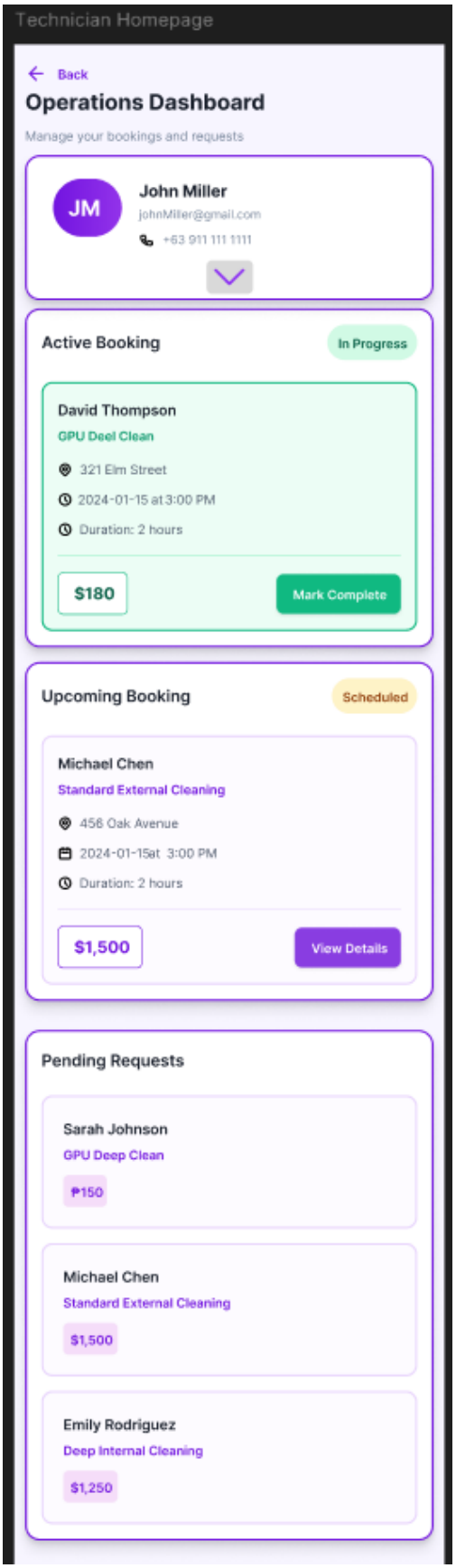
CLIENT Booking DETAILS Page (Complete)



CLIENT Booking DETAILS Page (No-Show)

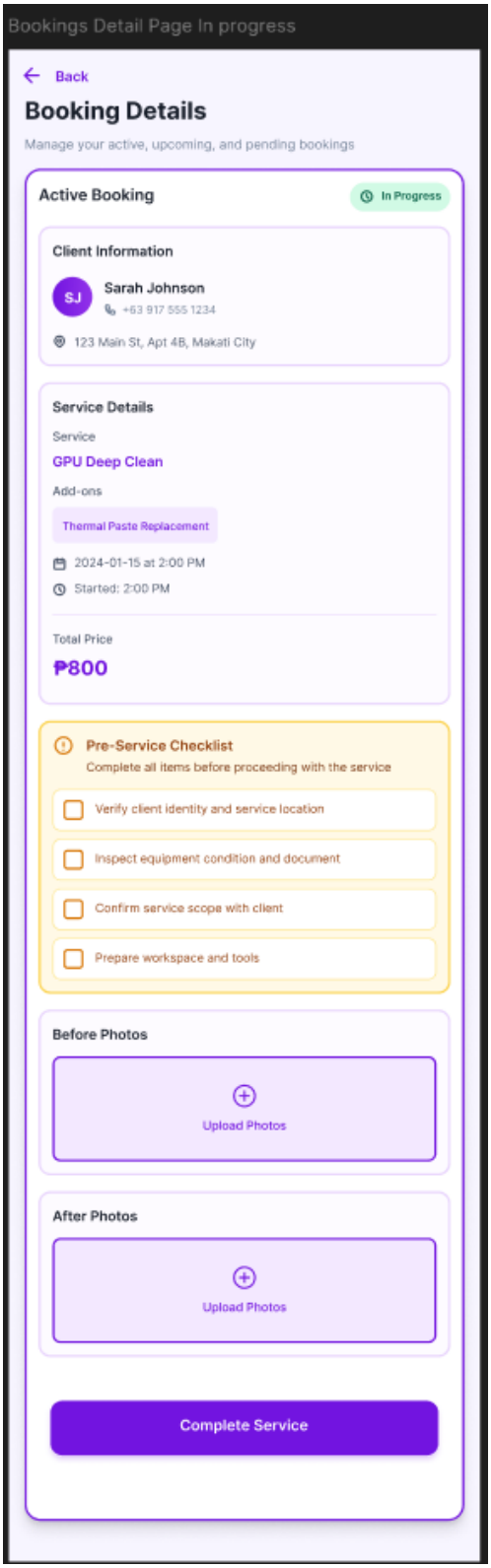


TECHNICIAN Homepage


Technician Homepage
Back
Operations Dashboard
Manage your bookings and requests
JM
John Miller
johnMiller@gmail.com
+63 911 111 1111
Active Booking
In Progress
David Thompson
GPU Deel Clean
321 Elm Street
2024-01-15 at 3:00 PM
Duration: 2 hours
$180
Mark Complete
Upcoming Booking
Scheduled
Michael Chen
Standard External Cleaning
456 Oak Avenue
2024-01-15at 3:00 PM
Duration: 2 hours
$1,500
View Details
Pending Requests
Sarah Johnson
GPU Deep Clean
₱150
Michael Chen
Standard External Cleaning
$1,500
Emily Rodriguez
Deep Internal Cleaning
$1,250

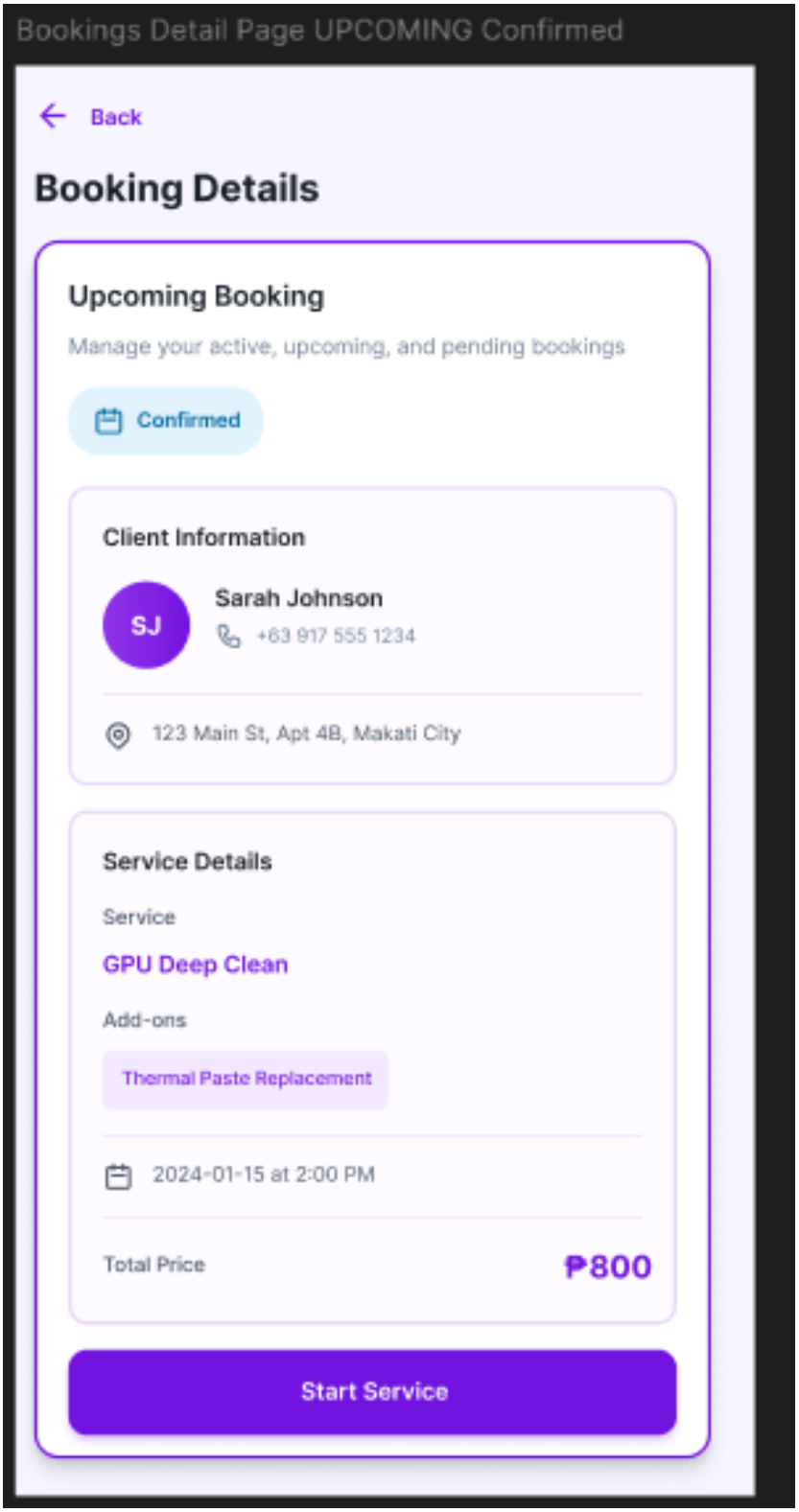
TECHNICIAN Bookings DETAIL Page (In Progress)

/Active Books/Checklists/Complete Service/



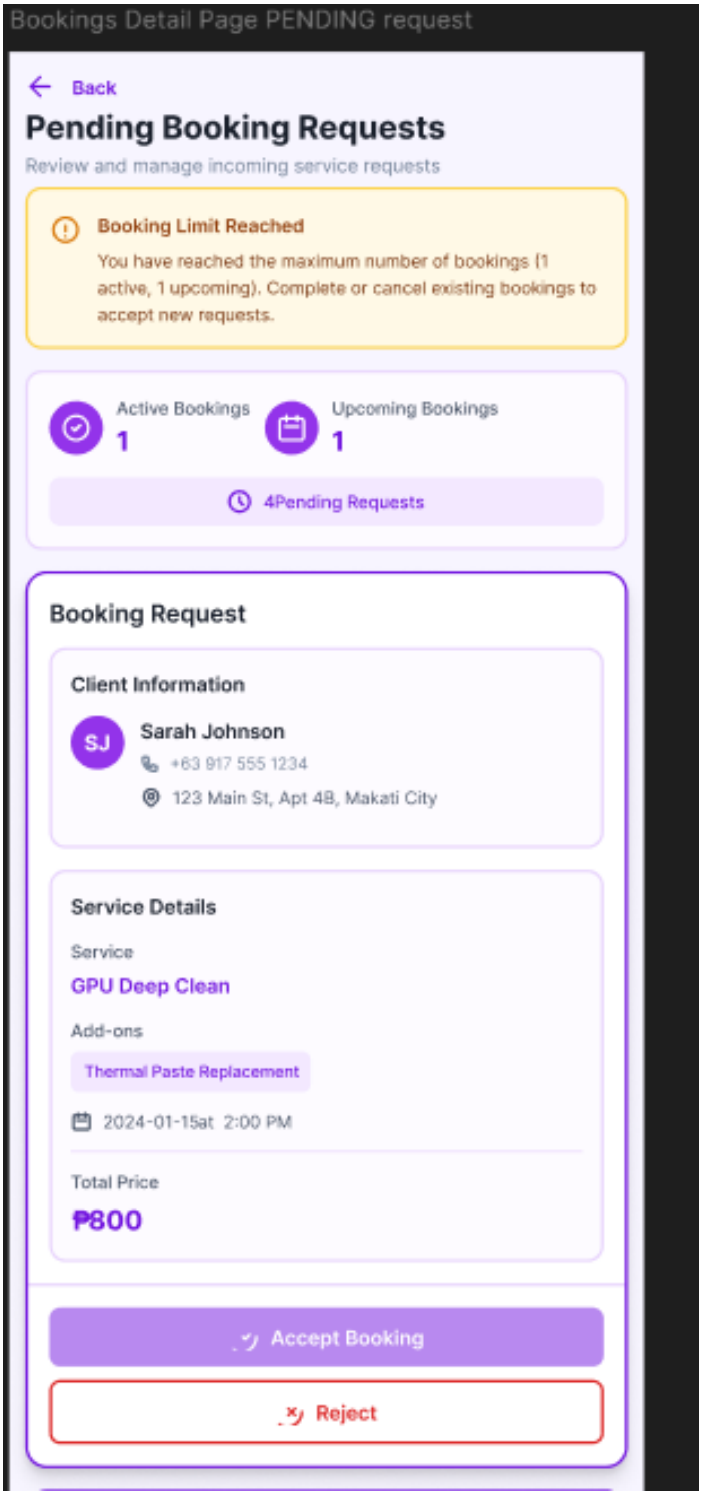
TECHNICIAN Bookings DETAIL Page (Confirmed)

/Active Books / Book Info/Start Service/No-Show/



TECHNICIAN Bookings DETAIL Page (Pending Request)

/Active Books / Book Info/ Accept or Reject /



Mobile-Specific Features:

- Touch-optimized buttons (minimum 44x44dp for accessibility compliance)
- Pull-to-refresh for booking list and status updates
- Swipe actions for quick cancel (Pending only)
- Offline caching for service images and static assets
- Bottom navigation for primary sections:
    1. Home
    2. My Bookings
    3. Create Booking
    4. Profile

Design System:

- **Colors:**
    - Primary: #2563EB
    - Secondary: #7C3AED
    - Success: #10B981
    - Error: #EF4444
    - Warning (Optional): #F59E0B
- **Typography:**
    - Inter (Web)
    - Default Android typography (Roboto) or custom Inter
    - Clear hierarchy (H1, H2, Body, Caption)
- **Spacing:**
    - 8px grid system
    - Consistent vertical rhythm
    - Standardized padding (16px default)
- **Components:**
    - Primary / Secondary Buttons

    - Status Badges (Pending, Confirmed, In Progress, Completed, Cancelled, No-Show)
    - Form Inputs
    - Cards for bookings
    - Modals for confirmation
    - Alert banners for errors

- **Responsive Strategy:**
    - Mobile-first
    - Breakpoints: 640px, 768px, 1024px
    - Grid-based layout for desktop dashboard

8.0 PLAN

8.1 Project Timeline

Phase 1: Planning & Design (Week 1-2)

Week 1: Requirements & Architecture

Day 1-2:

- Finalize functional requirements
- Finalize non-functional requirements

Day 3-4:

- Define service compatibility rules
- Define booking lifecycle states
- Define workload rule enforcement logic

Day 5-7:

- Define service compatibility rules
- Define booking lifecycle states
- Define workload rule enforcement logic

Week 2: Detailed Design

Day 1-2:

- Finalize ERD
- Define database constraints and relationships

Day 3-4:

- Complete API endpoint specifications

Day 5-6:

- UI/UX wireframes (Client + Technician)

Day 7:

- UI/UX wireframes (Client + Technician)

Phase 2: Backend Development (Week 3-4)

Week 3: **Backend Foundation**

Day 1:

- Spring Boot project setup
- Dependency configuration

Day 2:

- Database configuration (PostgreSQL via Supabase)
- Entity models and relationships

Day 3:

- JWT authentication implementation

Day 4:

- User registration, login, verification endpoints

Day 5:

- Service & Add-on management endpoints

Week 4: Core Features

Day 1:

- Booking creation endpoint

Day 2:

- Add-on compatibility validation logic

- Day 3: Time slot conflict prevention logic

Day 4:

- Technician workload enforcement (1 active + 1 upcoming)

Day 5:

- Lifecycle state transitions validation
- Checklist & photo upload validation

Phase 3: Web Application (Week 5-6)

Week 5: **Frontend Foundation**

Day 1:

- React + TypeScript setup

Day 2:

- Authentication pages

Day 3:

- Service catalog display

Day 4:

- Booking creation form

Day 5:

- Booking history & status tracking

Week 6: **Complete Web Features**

Day 1:

- Technician dashboard (Pending, Confirmed, In Progress)

Day 2:

- Checklist completion & photo upload UI

Day 3:

- Admin verification dashboard

Day 4:

- Responsive design polish

Day 5:

- Integration testing with backend

Phase 4: Mobile Application (Week 7-8)

Week 7: **Android Foundation**

Day 1:

- Android Studio setup
- Project structure

Day 2:

- Authentication screens

Day 3:

- Service browsing & booking creation

Day 4:

- Booking list & status tracking

Day 5:

- API service integration

Week 8: **Complete Mobile App**

Day 1:

- Technician view & checklist UI

Day 2:

- Before/After photo integration

Day 3:

- UI polish and gesture implementation

Day 4:

- Device testing

Day 5:

- APK build & documentation

Phase 5: Integration & Deployment (Week 9-10)

Week 9: **Integration Testing**

Day 1:

- End-to-end booking flow testing

Day 2:

- Workload rule validation testing

Day 3:

- Security validation (JWT, role access)

Day 4:

- Performance testing

Day 5:

- Bug fixing and documentation updates

Week 10: Deployment

Day 1:

- Backend deployment (Render)

Day 2:

- Web deployment (Render or Vercel)

Day 3:

- Supabase configuration (production DB & storage)

Day 4:

- Final system validation

Day 5:

- Final submission & documentation

Milestones:

- **M1 (End Week 2):** All design documents complete
- **M2 (End Week 4):** Backend API fully functional
- **M3 (End Week 6):** Web application complete
- **M4 (End Week 8):** Mobile application complete
- **M5 (End Week 10):** Full system deployed and integrated

Critical Path:

1. Authentication system (Week 3)
2. Product catalog API (Week 3-4)
3. Shopping cart functionality (Week 4)
4. Checkout process (Week 6)
5. Cross-platform testing (Week 9)

Risk Mitigation:

1. Implement core booking flow first
2. Enforce server-side validation early
3. Test workload & slot conflict logic before UI polish
4. Keep stable deployment branch
5. Validate lifecycle transitions at backend level